

使用 demo

- 1. 同步请求流程
- 2. 服务A (client) 状态说明
- 3. 服务A (Client) 接入样...
- 4. 服务B (server) 状态...
- 5. 服务B (server) 接入...
- 6. 跨线程请求流程

1 链路追踪中的基本概念

- 1.1 RPC 中的角色与过程：...
- 1.2 调用链路中的标记信息...
- 1.3 埋点上报的单位：Span
- 1.4 用户自定义埋点：Anno...

- 1. 产品形态
- 2. API
- 3. 技术方案
 - 3.1. 包依赖方案
 - 3.2. 压测标识传递方案
 - 3.2.1. 存储类中间件client...
 - 3.2.2. RPC/HTTP/MQ服务...
 - 3.2.2.1. 序列化反序列化...
 - 3.2.3. 使用约束
 - 3.3. Mtthrift服务数据传递方案
 - 3.4. HTTP服务数据传递方案
 - 3.5. MQ数据传递方案
- 5. 线下测试/灰度测试/应急响应 方...
- 5.1. 灰度方案
 - 5.1.1. 存储类中间件
- 5.2. 数据清理方案

- 一、使用说明
 - 1. 组件依赖
 - 2. 采样策略
 - 3. 链路状态

二、接口设计

mtrace

🔗 AI 速览

Mtrace是美团自研的分布式链路追踪系统，基于Google Dapper理念设计，通过在各中间件中埋点采集调用链信息，帮助快速定位性能瓶颈和故障。系统以**traceld**唯一标识整条调用链，以**spanId**标记各环节顺序，结合client/server端的埋点上报（clientSend/serverRecv等），形成完整调用拓扑。支持自定义**Annotation**添加关键信息便于检索，通过**Context**实现跨服务参数传递，满足压测、灰度等场景需求。提供Java SDK，支持同步/异步/跨线程埋点，封装线程池保障上下文传递，并兼容主流中间件自动集成。数据采用采样上报策略，异常链路强制采集，兼顾性能与可观测性。**全链路压测**、**慢查询分析**、**跨线程追踪**是其核心能力，适用于复杂微服务架构下的问题诊断与容量评估。

Mtrace系统如何通过traceld和spanId实现分布式调用链的追踪和定位？

Mtrace系统在跨线程或异步调用场景下如何保证上下文信息的传递和一致性？

🎨 风格切换 ▾

💬 继续提问

📄 🔄 | 🌟 💬 ...

×

Mtrace 是美团自研的分布式链路追踪系统，包括嵌入系统的埋点工具、数据收集、分析、存储、展示等一系列服务，其设计理念基于 [Google Dapper: 分布式链路追踪系统架构](#)。

Mtrace 通过在各个中间件中埋点，可以采集和标记每个分布式组件交互时的一些必要信息（调用方法，调用时长，调用结果等），进而提示调用链的全貌、状态、异常等信息，也利于分析耗时瓶颈及通过日志追查问题。

i 下文中的例子均以 Mtrace Java Sdk 中的概念表示，如果对代码描述有疑问，可以参考 [Mtrace sdk-接口文档 \(Java\)](#)。

使用 demo

1. 同步请求流程



2. 服务A (client) 状态说明

1. 状态查询接口

2. 设置埋点属性接口

3. 上下文接口

4. 埋点状态变更接口

5. 其它接口

6. 手动强制设置 appKey

三、跨线程埋点方法

1. 线程封装

2. 线程池封装

3. 线程池封装 (javaagen...

XFrame开发框架

MDP开发框架

Tomcat war包部署

Springboot Jar包部署

运行时动态attach mt...

4. 跨线程传递 ThreadLo...

5. Springboot @Async...

四、AOP 方法埋点使用教程

五、参数配置

1.配置项说明

2. 关于强制上报的基础...

3. 关于采样配置的说明

4. 配置生效验证

六、使用 demo

1. 同步请求流程

2. 服务A (client) 状态说明

3. 服务A (Client) 接入样...

4. 服务B (server) 状态...

5. 服务B (server) 接入...

6. 跨线程请求流程

1.5 使用实现了埋点功能的...

2 工作原理

2.1 串行任务模型

2.2 并行任务模型

2.3 异步模型

3 数据采集

3.1 数据采样

阶段A1 (非必需)

代码块

```
1 TraceParam param = new TraceParam("method");
2 param.setInfraName("infraName");
3 param.setRemoteAppKey("remote appkey");
4 param.setRemoteIp("127.0.0.1");
```

需要在埋点前构建一个数据结构存储相关信息的时候可以使用该变量，如果不需要可以不使用，在后面的埋点过程中装填。

TraceParam 属性	Desc	网络传递字段
traceld	server端需要把网络中传过来的traceld填进来	true
spanId	server端需要把网络中传过来的spanId填进来	true
spanName	类名 + “.” + 方法名	
remoteAppKey	远端的appkey	
remoteIp	远端的ip	
infraName	中间件名称 EG : mtthrift, cellar, zebra	
version	中间件版本 EG : xx.xx.xx	
packageSize	包大小, 单位字节	
debug	server端需要把网络中传过来的debug字段填进来	true
foreverContext	server端需要把网络中传过来的foreverContext字段填进来	true
oneStepContext	server端需要把网络中传过来的oneStepContext字段填进来	true

阶段A2

代码块

```
1 // 以下是埋点的三种方式
2 Tracer.clientSend(param); // 方式1 参数是TraceParam
3 Tracer.clientSend("Class.method"); // 方式2 参数是 类名.方法名
4 Tracer.clientSend(); // 方式3 无参数
```

- `Tracer.clientSend(param)` 把`TraceParam`当作参数传入埋点开始接口，会把`TraceParam`设置的参数装填到埋点上下文中
- `Tracer.clientSend("Class.method")` 埋点开始接口，会自己构建一个埋点上下文，名称是`Class.method`
- `Tracer.clientSend()` 埋点开始接口，会自己构建一个空的埋点上下文，如果后面到埋点结束也没有添加名称，则不会上报信息

阶段A3

3.2 目前的数据上报/数据采...

阿里鹰眼

概述

基本原理

系统架构

主要功能

发现瓶颈、容量预估

[图片]

StreamLib 提升流计算性能

监控系统确定性保证

个性化需求

定位故障原因

系统优化

存储优化

从被动报警到主动发现

代码块

```
1 ITracer tracer = Tracer.getClientTracer(); // 获取当前埋点对应的tracer
2 tracer.setSize(100); // 设置接收到的请求包大小
3 tracer.setStatus(Tracer.STATUS.SUCCESS); // 设置请求的返回结果
```

1. 获取埋点上下文封装对象

2. 如果能获取到上下文信息，则设置请求包大小

3. 如果能获取到上下文信息，则设置请求返回状态

setName	设置请求接口名
setStatus	设置请求状态
setInfraName	设置中间件名称
setSize	设置请求包大小
setAsync	设置是否为异步调用
setSample	设置是否采样标识
putContext	设置传递上下文数据
putLocalOneStepContext	设置本地传递一级上下文数据
putRemoteOneStepContext	设置远程传递一级上下文数据
info	设置调用信息属性
addAnnotation	设置调用信息属性（会建立索引）

阶段A4

代码块

```
1 Tracer.clientRecv();
```

请求埋点结束

3. 服务A (Client) 接入样例

代码块

```
1 TraceParam param = new TraceParam("B.call()");
2 param.setInfraName("mtthrift");
```

```
3 param.setRemoteAppKey("com.meituan.service.B");
4 param.setRemoteIp("10.11.24.132");
5
6 Tracer.clientSend(param); // 埋点开始
7
8 ITracer tracer = Tracer.getClientTracer(); // 获取当前埋点对应的tracer
9 tracer.setSize(100);
10 try {
11     Response res = B.call(...); // 中间件调用
12     tracer.setStatus(Tracer.STATUS.SUCCESS);
13 } catch (Exception e) {
14     tracer.setStatus(Tracer.STATUS.EXCEPTION);
15 } finally {
16     Tracer.clientRecv(); // 埋点结束
17 }
18
```

4. 服务B (server) 状态说明

阶段B1

代码块

```
1 TraceParam param = new TraceParam("method");
2 param.setInfraName("infraName");
3 param.setRemoteAppKey("remote appkey");
4 param.setRemoteIp("127.0.0.1");
5
6 /*
7  * 如果网络中传递过来了trace信息，需要装填网络传递过来的trace信息
8  */
9 param.setTraceId(traceId);
10 param.setSpanId(spanId);
11 param.setDebug(debug);
12 param.setForeverContext(foreverContext);
13 param.setOneStepContext(oneStepContext);
```

- traceId 请求唯一标识
- spanId 请求位置标识
- debug 是否采集存储
- foreverContext 自定义传递上下文
- oneStepContext 自定义传递一级的上下文

阶段B2

代码块

java

```
1 // 以下是埋点的三种方式
2 Tracer.serverRecv(param); // 方式1 参数是TraceParam
3 Tracer.serverRecv("Class.method"); // 方式2 参数是 类名.方法名
4 Tracer.serverRecv(); // 方式3 无参数
```

阶段B3

代码块

java

```
1 ITracer tracer = Tracer.getServerTracer(); // 获取当前埋点对应的tracer
2 tracer.setSize(123); // 设置接收到的请求包大小
3 tracer.setStatus(Tracer.STATUS.SUCCESS); // 设置请求的返回结果
4 tracer.info("serialize", "12", System.currentTimeMillis()); // 设置中间件序列化所耗费的时间, KV结构, 可以自行定义
```

阶段B4

代码块

java

```
1 Tracer.serverSend();
```

5. 服务B (server) 接入样例

代码块

java

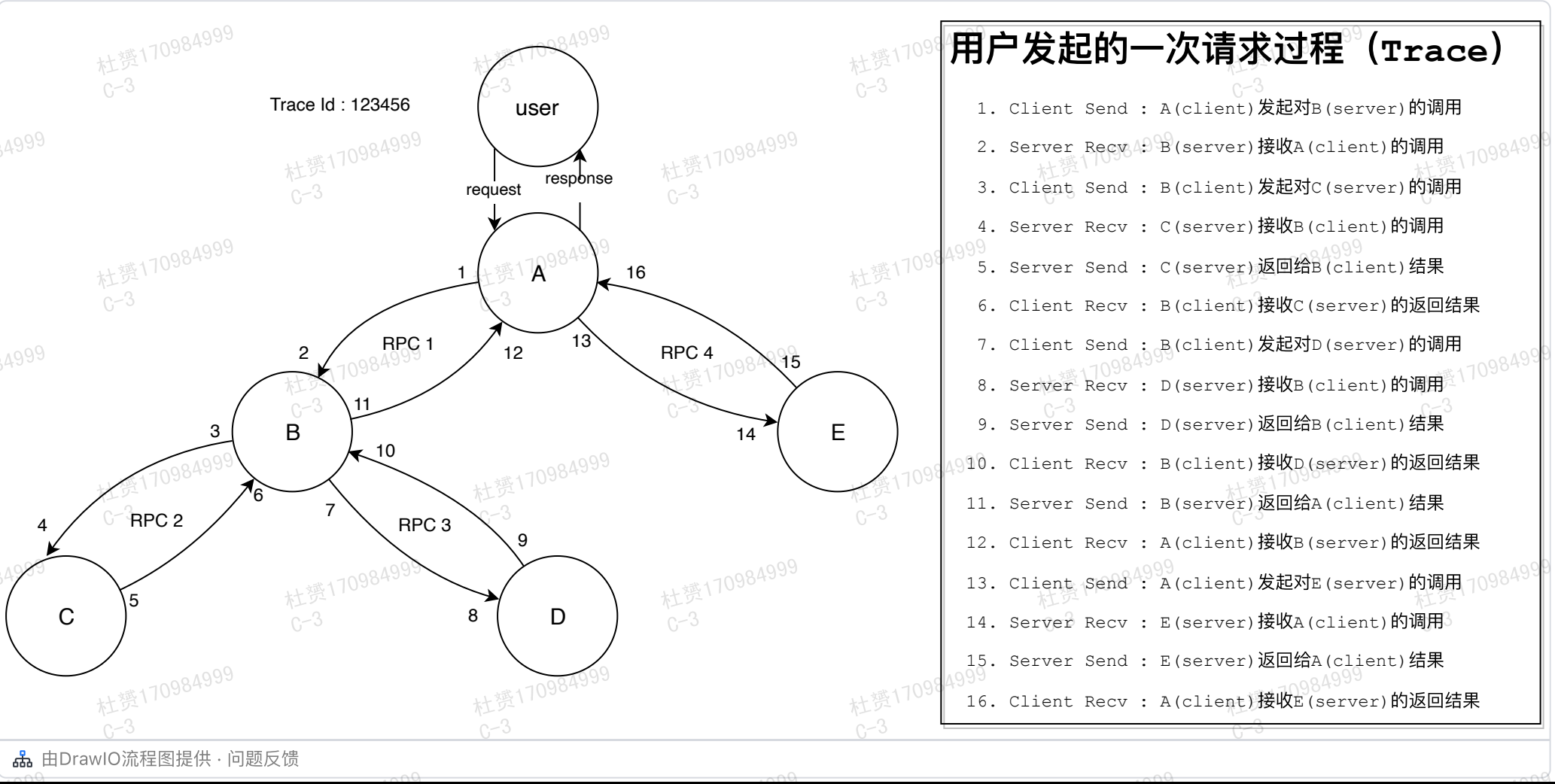
```
1 TraceParam param = new TraceParam("B.call()");
2 param.setInfraName("mtthrif");
3 param.setRemoteAppKey("com.meituan.service.A");
4 param.setRemoteIp("127.0.0.1");
5
6 param.setTraceId(traceId);
7 param.setSpanId(spanId);
8 param.setDebug(debug);
9 param.setForeverContext(foreverContext);
10 param.setOneStepContext(oneStepContext);
11
12 // 请求接收处理线程
13 Tracer.serverRecv(param); // 埋点开始
14
15 ITracer tracer = Tracer.getServerTracer(); // 获取当前埋点对应的tracer
16 tracer.setSize(111);
17 tracer.info("serialize", "12", System.currentTimeMillis());
18 tracer.info("getConnection", "8", System.currentTimeMillis());
19 try {
20 // 业务端逻辑处理部分
```

```
21     tracer.setStatus(Tracer.STATUS.SUCCESS);
22 } catch (Exception e) {
23     tracer.setStatus(Tracer.STATUS.EXCEPTION);
24 } finally {
25     Tracer.serverSend(); //埋点结束
26 }
27
```

6. 跨线程请求流程

1 链路追踪中的基本概念

下面以一个简单的分布式系统来描述一下和 Mtrace 相关的基本概念与工作原理。



1.1 RPC 中的角色与过程：Client、Server、Trace

如图可以看到，user 发起的请求过程中一共包含了四次 RPC 调用，我们称发起 RPC 调用的角色为 **Client**，接收请求并返回结果的角色为 **Server**。

以 RPC1 为例，A 在此次 RPC 调用中的角色为 Client，B 的角色为 Server。

同时，在每一次完整的 RPC 调用中，都包含了以下四个过程：

- Client Send：客户端发起请求
- Server Recv：服务端接收请求
- Server Send：服务端返回结果

- Client Recv：客户端接收结果

这四个交互过程就是 Mtrace 主要的埋点过程。

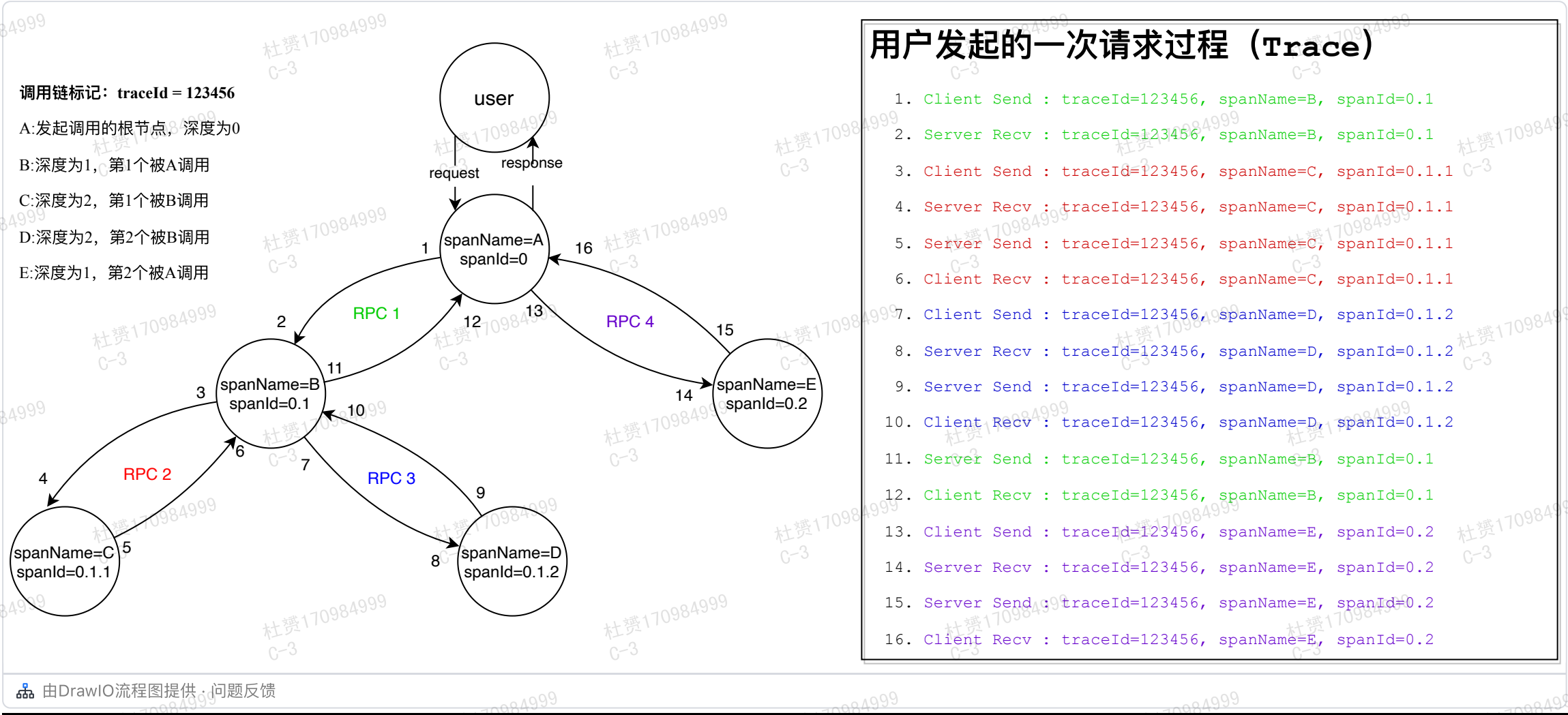
将所有的 1-16 共四次 RPC 调用结合起来，就构成了一次完整的请求过程，即 **Trace**。

一般可以认为每次用户（浏览器，APP）发起请求，都是一条全新的调用链。

1.2 调用链路中的标记信息：TraceId、SpanId

Mtrace 会给每条完整的链路一个唯一 id 作为标记。这个标记在整个 Trace 过程中都会携带，这样，在 A、B、C、D、E 分别上报自己的埋点信息时，就可以通过 **traceid** 关联起一条完整的调用链路。

同时，Mtrace 还会给每个环节（A、B、C、D、E）一个标记(**spanName**)，表示调用的不同环节，并且用序号(**spanId**)表示调用的发起顺序。仍旧以例子说明：



在这个例子中，调用发起的顺序为 RPC1 > RPC2 > RPC3 > RPC4，对应到每次 RPC Call 的 server 端的 spanId 即 0.1 > 0.1.1 > 0.1.2 > 0.2。

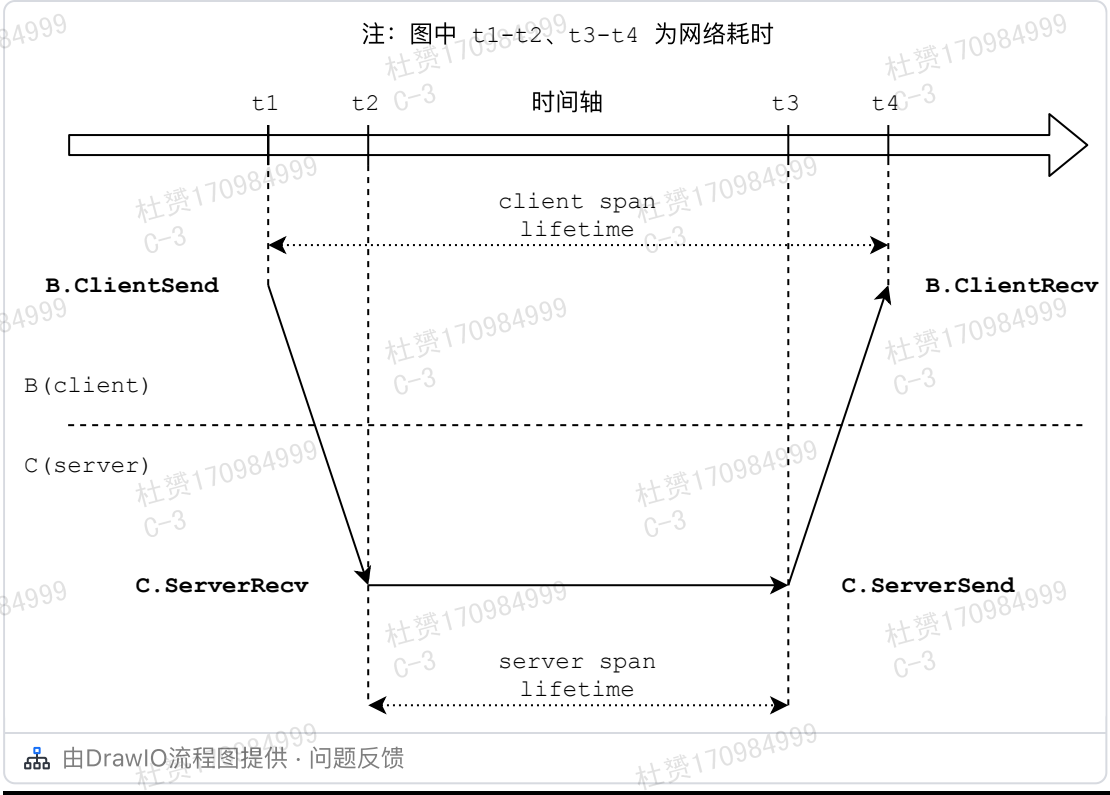
除了 traceId、spanName、spanId 之外，每次 rpc 调用还可以包含其它的交互信息，例如 appKey、local address、remote address、duration 等。

spanName 通常根据参与交互的双方角色不同稍有不同，比如 rpc 中 spanname 可以是类名+方法名，zebra spanname 可以是 sql，mafka 可以使用 topic + 消费组等等。

1.3 埋点上报的单位：Span

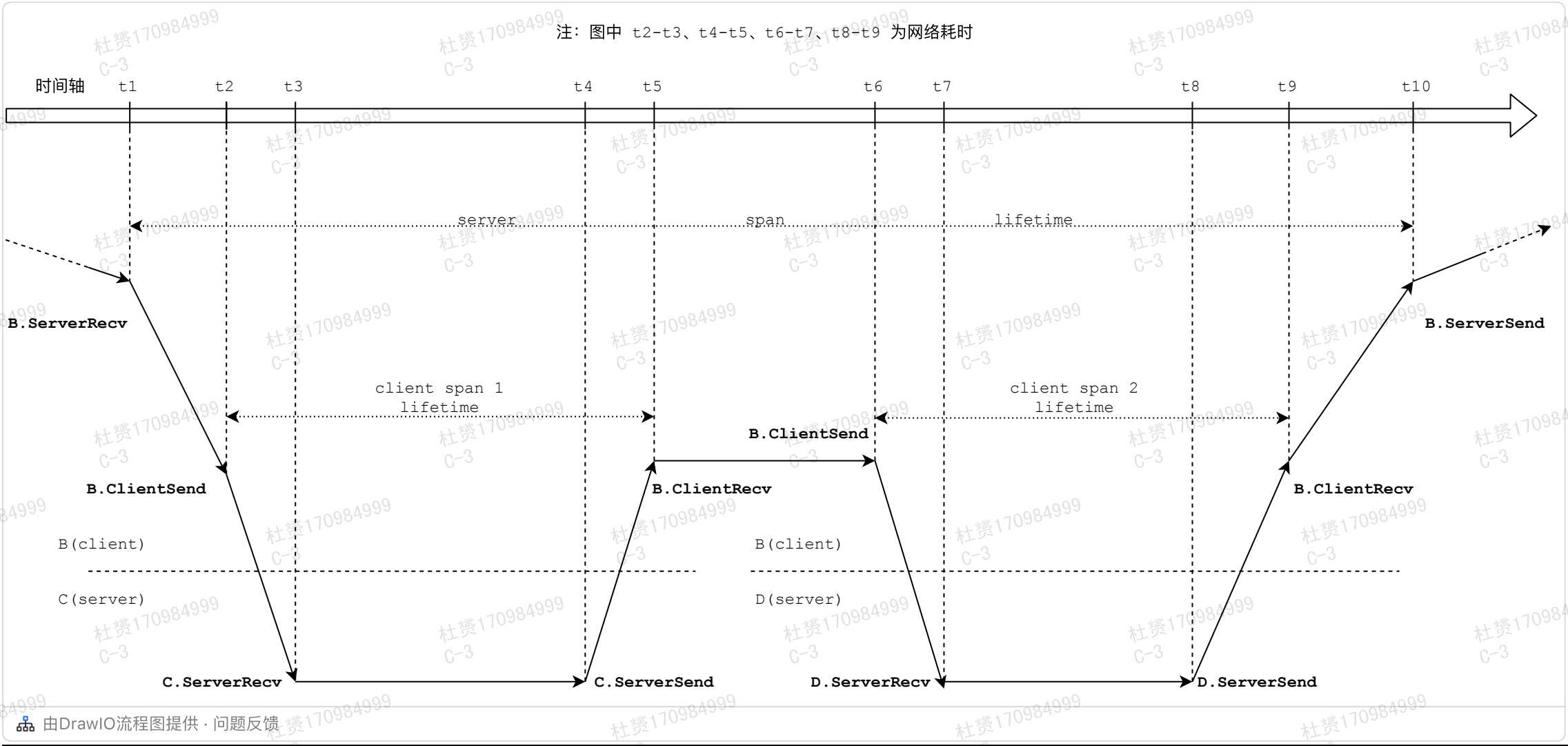
每次 RPC 调用的埋点由 server 和 client 参与，在调用完成后，server 和 client 会分别上报一个埋点信息，称为 **span**。

server 身份上报的 span 称为 serverSpan，client 身份上报的 span 称为 clientSpan。以上图中的 RPC2、B 分别举例：



B 发起 ClientSend 时，client span 的生命周期开始，同时也意味着 B 端埋点开始。B 调用 ClientRecv 时，RPC 过程结束，span 的生命周期也结束，此时会发起 span 的上报（异步）。C 端同理。

在某些 RPC 调用中，client 端是不方便埋点的（比如外部系统或者前端），因此信息展示时一般以 server 端为主，传递的 span 信息也是 server 端的信息。



图中的各个 span 及生命周期分别为：

- B-ServerSpan (t1 ~ t10)

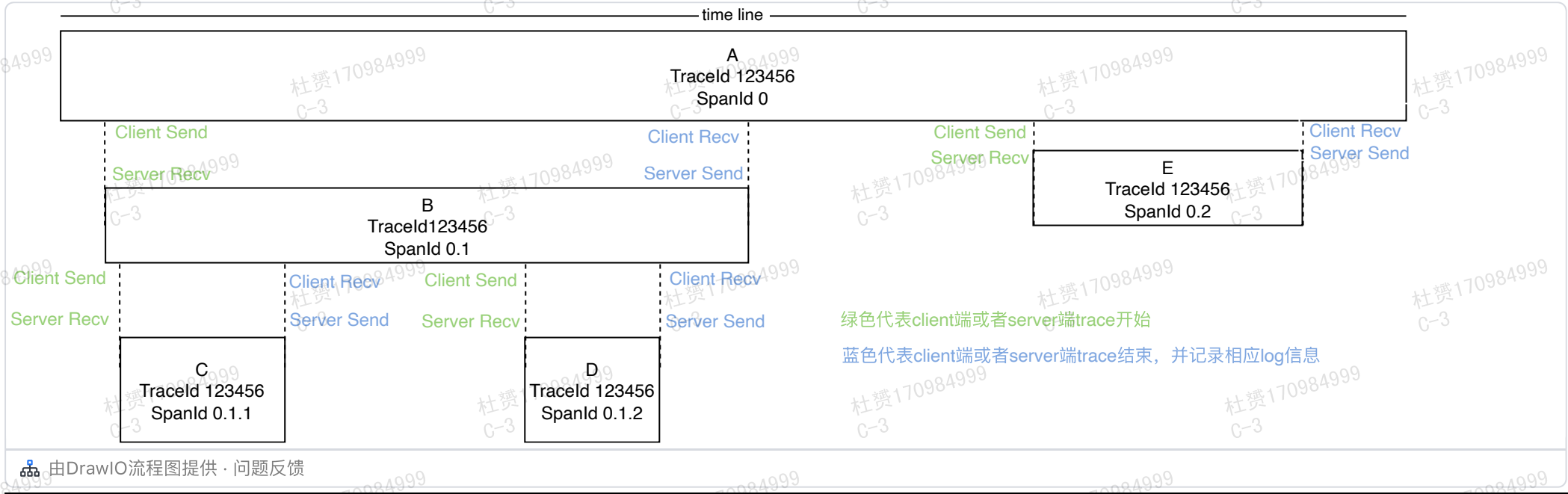
- B-ClientSpan1 (t2 ~ t5)
- C-ServerSpan (t3 ~ t4)
- B-ClientSpan2 (t6 ~ t9)
- D-ServerSpan (t7 ~ t8)

在一次 RPC 调用中，client span 的生命周期一般是要长于 server span 的 (B-ClientSpan1 > C-ServerSpan)

但是对同一个程序而言，埋点的顺序一定是先开启 server span，再开启 client span 的埋点 (B-ServerSpan > B-ClientSpan1)，如下所示：

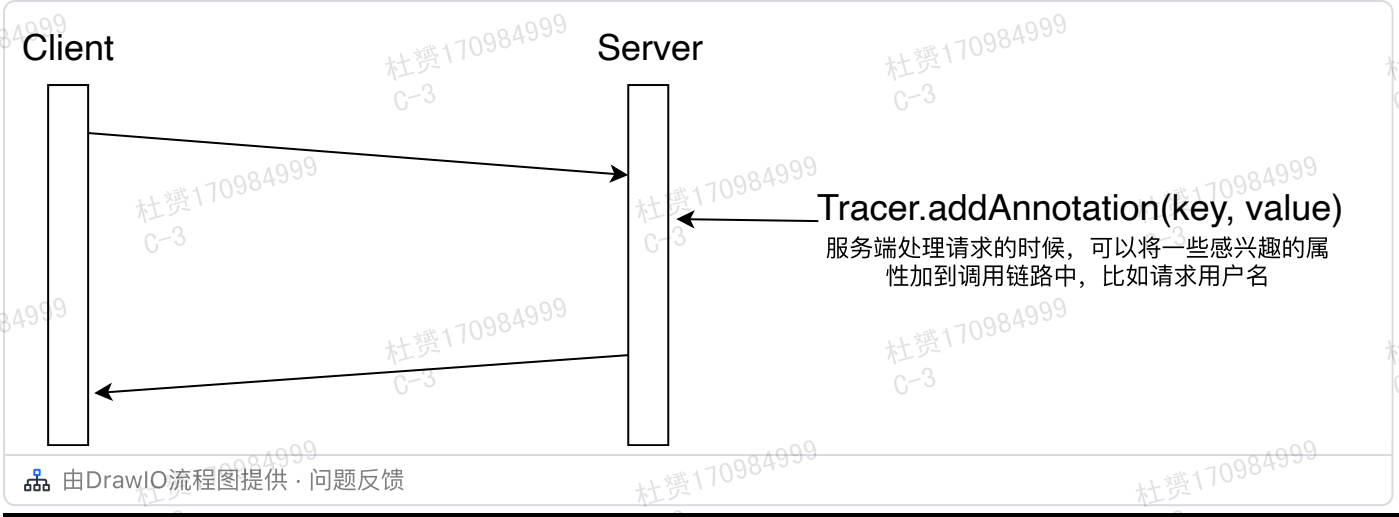
1. ServerRecv (B-ServerSpan begin)
2. ClientSend (B-ClientSpan1 begin)
3. ClientRecv (B-ClientSpan1 end)
4. ClientSend (B-ClientSpan2 begin)
5. ClientRecv (B-ClientSpan2 end)
6. ClientSend (B-ClientSpan3 begin)
7. ClientRecv (B-ClientSpan3 end)
8. (重复上述两步)
9. ServerSend (B-ServerSpan end)

下图展示了整个例子中的 server span 的生命周期及埋点顺序。



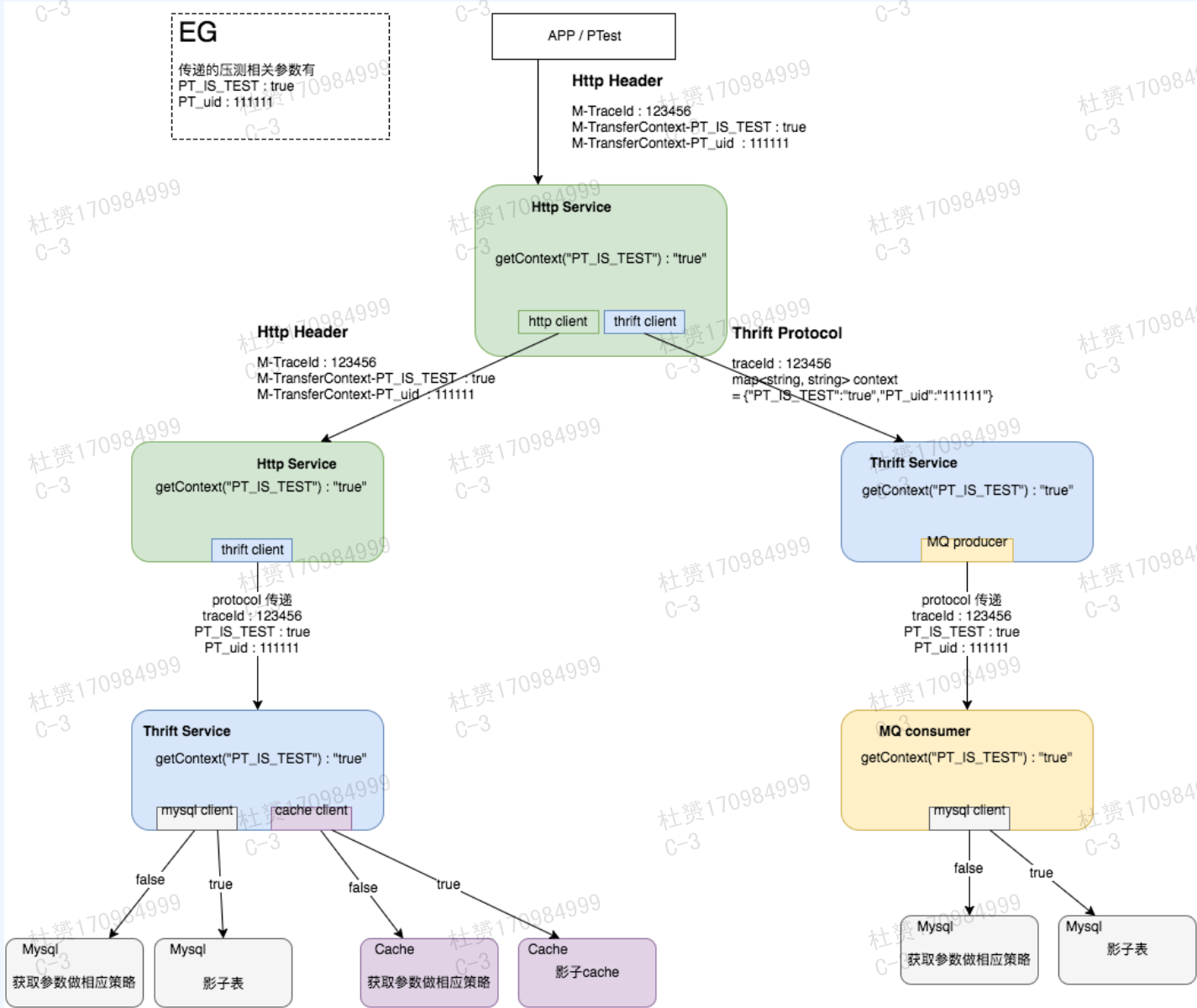
1.4 用户自定义埋点：Annotation、Context

为了更方便的定位问题链路，Mtrace 提供了自定义埋点功能。业务可以手动传递 key-value tag 到 span 中。比如给每一条trace加上一个uid={{用户名称}}，或者加上异常名称 exceptionName=NPE。添加自定义埋点信息 (Annotation) 后，在 Mtrace 平台上就可以用相关的表达式定位相关的链路了。



上下文 (**Context**) 的存储结构同样是 key-value map，和 Annotation 不同的地方在于，Annotation 主要用于属性的上报，而 Context 主要用于 server 和 client 的通信参数，传递的信息主要包括灰度标记、泳道信息等，在支持 [全链路压测](#)、服务鉴权等特殊场景中起到了不可替代的作用。

1. 产品形态



2. API

[Mtrace sdk-接口文档 \(Java\)](#)

注：

压测相关key命名规范需要以 **PT_** 为前缀。

比如：

Tracer.putContext("PT_USERID", "123456")

Tracer.getContext("PT_USERID")[Mtrace sdk-接口文档 \(Java\)](#)

3. 技术方案

全链路传递压测标识，各中间件通过读取压测标识来做相应逻辑判断

3.1. 包依赖方案

[Mtrace 拆包方案](#) 目前已使用 [inf-bom Release Notes](#) 统一依赖。包依赖关系见 [中间件依赖树](#)。

3.2. 压测标识传递方案

服务内部统一使用mtrace传递上下文参数，由mtrace提供统一的对外put，get接口。

RPC/HTTP中间件需要支持在网络协议传输中携带上下文字段，在发起请求时将本地的上下文传递给远端服务，远端服务在接收请求后，可以解析出上下文，然后将上下文写入mtrace结构中供业务端使用。

详细方案可参考：[Mtrace 跨中间件透明传递参数](#)

3.2.1. 存储类中间件client端获取标识的方法

Tracer.get(key)

判断是否是压测流量：Tracer.isTest()

3.2.2. RPC/HTTP/MQ服务传递标识的方法

需要调用mtrace 埋点API。

请求发送过程

获取调用Span：Span span = Tracer.getClientTracer().getSpan()，如果没有埋点的话可能为null

获取传递一级的上下文map结构：span.getRemoteOneStepContext()

获取一直传递的上下文map结构：span.getForeverContext()

通过RPC 协议层将上下文传递给后端服务

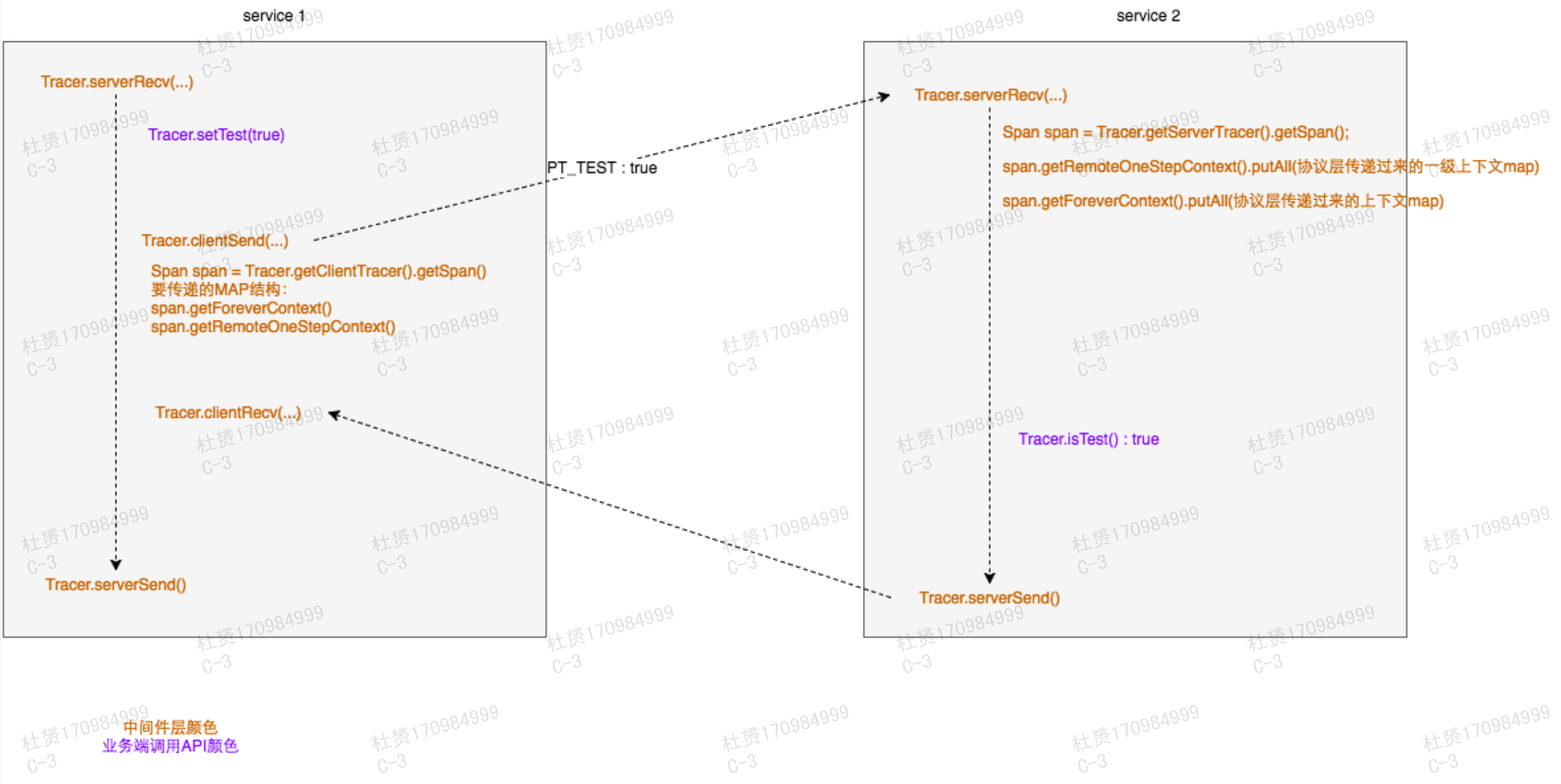
请求接收过程

通过协议层获取上下文信息

获取请求接收线程Span：Span span = Tracer.getServerTracer().getSpan();

赋值传递一级的上下文map结构：span.getRemoteOneStepContext().putAll(协议层传递过来的一级上下文map)

赋值一直传递的上下文map结构：span.getForeverContext().putAll(协议层传递过来的上下文map)



3.2.2.1. 序列化反序列化方法

各个中间件都会有基于自己协议的序列化反序列化方法用于传递上下文数据，对于没有自有协议的中间件， mtrace提供了一个基于thrift协议的序列化反序列化方式

代码块

Java

```
1 public class ThSerializer {
2     public static byte[] serialize(); // 基于ThreadLocal中的span结构进行序列化
3     public static byte[] serialize(final Span span); // 手动传入span 进行序列化
4     public static void deserialize(TraceParam param, byte[] header); // 反序列化上下文到TraceParam中
5 }
```

调用示例

代码块

Java

```
@Test
2
3     public void test() {
4
5         // 发送请求
6
7         Span span = Tracer.clientSend(new TraceParam("client test"));
```

```
8         String traceId = span.getTraceId();
9
10
11        String spanId = span.getSpanId();
12
13        byte[] bytes = ThSerializer.serialize(); // 序列化成bytes
14
15        logger.info("serialize : {}", bytes);
16
17        Tracer.clientRecv();
18
19        // 接收请求
20
21        TraceParam param = new TraceParam("server test");
22
23        ThSerializer.deserialize(param, bytes); // 反序列化bytes到TraceParam中
24
25        Assert.assertEquals(traceId, param.getTraceId());
26
27        Assert.assertEquals(spanId, param.getSpanId());
28
29        Tracer.serverRecv(param);
30
31        Tracer.serverSend();
32
33    }
```

3.2.3. 使用约束

- 传递的key value对不能有\t, \n 字符
- 传递key value对的数量最大为20
- key, value 的大小最大为256个字符

3.3. Mtthrift服务数据传递方案

[MTthrift 支持全链路压测设计文档](#)

3.4. HTTP服务数据传递方案

[HTTP 服务全链路压测方案](#)

3.5. MQ数据传递方案

[Mafka全链路压测方案](#)

[RabbitMQ/MTMQ 全链路压测方案](#)

5. 线下测试/灰度测试/应急响应 方案

5.1. 灰度方案

5.1.1. 存储类中间件

支持按表的维度设置读写是否偏移。具体方案是中间件支持表维度的开关配置，开关打开则表示需要偏移，开关关闭则表示不需要偏移。所以有四个排列组合：

- 某表开关打开：
- a. client已升级，则读写均偏移
 - b. client未升级，则读不到开关，所以读写原表

- 某表开关关闭：
- c. client已升级，则读写原表
 - d. client未升级，读写原表

这里需要额外关注的是b情况，如果某表需要偏移，那么读写该表的全部服务均需升级中间件

5.2. 数据清理方案

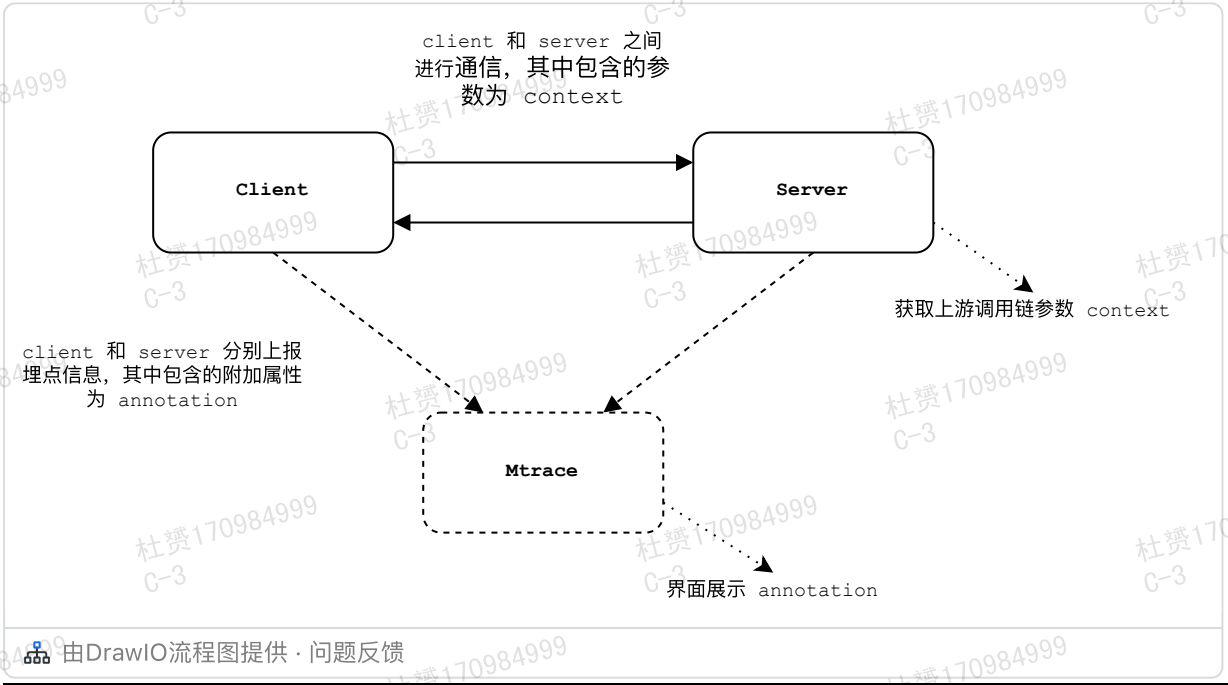
一些存储表不偏移，压测数据会写入原表；某些异常情况下，压测数据也会写入需偏移表的原表中。所以我们要具备清理异常数据的能力。

现有数据清理方案：

Cellar/Squirrel：设置压测数据的过期时间，通过过期逐出压测数据（**也要考虑异常情况下过期时间设置错误怎么办**）

Zebra：写入DB的压测数据有特征标识，通过改标识由DBA来清洗数据

MQ的积压数据：？



如果你是中间件开发者，通常需要关注的是 Context 的概念和用法。参见 [Mtrace 跨中间件透明传递参数](#)。

如果你是 Mtrace 使用方，通常需要关注的是 Annotation 的埋点和定位。参见 [Mtrace sdk-接口文档 \(Java\)](#)。

i Mtrace sdk-接口文档 (Java)-副本

目录

[使用 demo](#)

[1. 同步请求流程](#)

- 2. 服务A (client) 状态说明
- 3. 服务A (Client) 接入样例
- 4. 服务B (server) 状态说明
- 5. 服务B (server) 接入样例
- 6. 跨线程请求流程

- 1 链路追踪中的基本概念
 - 1.1 RPC 中的角色与过程：Client、Server、Trace
 - 1.2 调用链路中的标记信息：Traceld、SpanId
 - 1.3 埋点上报的单位：Span
 - 1.4 用户自定义埋点：Annotation、Context

1. 产品形态

2. API

3. 技术方案

- 3.1. 包依赖方案
- 3.2. 压测标识传递方案
 - 3.2.1. 存储类中间件client端获取标识的方法
 - 3.2.2. RPC/HTTP/MQ服务传递标识的方法
 - 3.2.2.1. 序列化反序列化方法

- 3.2.3. 使用约束
- 3.3. Mtthrift服务数据传递方案
- 3.4. HTTP服务数据传递方案
- 3.5. MQ数据传递方案

5. 线下测试/灰度测试/应急响应 方案

- 5.1. 灰度方案
 - 5.1.1. 存储类中间件

5.2. 数据清理方案

一、使用说明

- 1. 组件依赖
- 2. 采样策略
- 3. 链路状态

二、接口设计

- 1. 状态查询接口
- 2. 设置埋点属性接口
- 3. 上下文接口
- 4. 埋点状态变更接口
- 5. 其它接口
- 6. 手动强制设置appKey

三、跨线程埋点方法

- 1. 线程封装
- 2. 线程池封装

3. 线程池封装 (javaagent 方式)

XFrame开发框架

MDP开发框架

Tomcat war包部署

Springboot Jar包部署

运行时动态attach mtrace agent

4. 跨线程传递 ThreadLocal 变量

5. Springboot @Async透传mtrace信息

四、AOP 方法埋点使用教程

五、参数配置

1. 配置项说明

2. 关于强制上报的基础组件配置的说明 (http、mtthrift、zebra)

3. 关于采样配置的说明

4. 配置生效验证

六、使用 demo

1. 同步请求流程

2. 服务A (client) 状态说明

3. 服务A (Client) 接入样例

4. 服务B (server) 状态说明

5. 服务B (server) 接入样例

6. 跨线程请求流程

1.5 使用实现了埋点功能的中间件

2 工作原理

2.1 串行任务模型

2.2 并行任务模型

2.3 异步模型

3 数据采集

3.1 数据采样

3.2 目前的数据上报/数据采样策略

阿里鹰眼

概述

基本原理

系统架构

主要功能

发现瓶颈、容量预估

[图片]

StreamLib 提升流计算性能

监控系统确定性保证

个性化需求

定位故障原因

- 系统优化
- 存储优化
- 从被动报警到主动发现

一、使用说明

观看此使用说明前，建议先阅读 [Mtrace-01-用户手册](#) 了解基本概念。

1. 组件依赖

推荐使用最新版 [inf-bom](#) 统一依赖。包依赖关系见 [INF组件依赖树](#) 。变更记录见 [Mtrace ChangeLog \(Java\)](#) 。

- ❗ 如果自行指定依赖，需要保证所有 mtrace 相关包的版本前三位相同，例如 1.1.24.x
- 个别包使用低版本 mtrace 会造成服务抛出 **NoSuchMethodError** 或 **NoClassDefFoundError** 等异常！

```
^ pom.xml
1  <dependencyManagement>
2      <dependencies>
3          <dependency>
4              <groupId>com.sankuai</groupId>
5              <artifactId>inf-bom</artifactId>
6              <version>${inf-bom.version}</version>
7              <type>pom</type>
8              <scope>import</scope>
9          </dependency>
10     </dependencies>
11 </dependencyManagement>
12 <dependency>
13     <groupId>com.meituan.mtrace</groupId>
14     <artifactId>mtrace</artifactId>
15     <!--<version>${mtrace.version}</version>-->
16 </dependency>
17 <dependency>
18     <groupId>com.meituan.mtrace</groupId>
19     <artifactId>mtrace-api</artifactId>
20     <!--<version>${mtrace.version}</version>-->
21 </dependency>
22 <dependency>
23     <groupId>com.meituan.mtrace</groupId>
24     <artifactId>idl-mtrace</artifactId>
25     <!--<version>${mtrace.version}</version>-->
26 </dependency>
```

- ❗ 如果你是中间件开发者，不需要验证实际的 mtrace 上报功能，可以不依赖 mtrace 包，或者使用 `<scope>provided</scope>` 依赖模式。

2. 对于存储类中间件 (如 zebra, mafka) 来说, 因为没有server端, 所以只需要埋点client端即可。
3. 如果你间接依赖了 mtrace (比如使用了 mtthrift), mtrace 的版本应该以中间件中指定的版本为准。举例来说, 假设目前最新版 ORG sdk 依赖的 mtrace 版本为 1.1.16.2, 则建议使用 ORG sdk 的用户降级 mtrace 到 1.1.16.2 版本

2. 采样策略

mtrace java sdk 的采样策略按照优先级排序如下:

- 如果某个环节状态异常 (比如 TIMEOUT、EXCEPTION、HTTP_4XX、HTTP_5XX), 则一定会采集
- 对于其它埋点数据默认使用采样存储的方式。默认采样率为 0.01% 或 5次/分钟

注: 强制采集只对下游调用链生效, 其它兄弟子树可能已经上报完毕, 不保证整条链路会全部采集。

关于异常链路的强制采集功能参见 [Mtrace 异常链路存储](#)。

3. 链路状态

对于 thrift 服务, 使用 success、timeout、exception、drop 来对描述一次 RPC 调用的结果; 对 http 服务, 业界更普遍使用状态码来描述: 2xx、3xx、4xx、5xx。

埋点状态	所属协议	真实值	备注
Tracer.SUCCESS	thrift 等	0	
Tracer.EXCEPTION	thrift 等	1	该状态会被强制采集
Tracer.TIMEOUT	thrift	2	该状态会被强制采集
Tracer.DROP	thrift	3	该状态会被强制采集
Tracer.HTTP_2XX	http	12	
Tracer.HTTP_3XX	http	13	
Tracer.HTTP_4XX	http	14	
Tracer.HTTP_5XX	http	15	该状态会被强制采集

二、接口设计

1. 状态查询接口

Tracer.id()

返回值: String

获取当前线程的 trace id, 如果没有 trace 信息 (当前线程非请求处理线程, 或者不在埋点中) 则返回 null

Tracer.getServerSpanId()

返回值: String

获取当前线程的 server span id, 如果没有 trace 信息 (当前线程非请求处理线程, 或者不在埋点中) 则返回 null

Tracer.getAppKey()

返回值：String

服务端获取该次调用远程客户端的 appkey，没有返回NULL

2. 设置埋点属性接口

- !
1. 埋点属性最多100个。

2. 如果不需要使用该 key 来在mtrace管理平台通过表达式进行查询，请使用 Tracer.info 接口。

3. 如果 value 长度超过 100，或 key 长度超过 80，则不会存 es 索引。

4. 业务方请勿使用 MT、MTrace 做 key 的前缀，避免命名冲突。

5. 参数请勿传 null。

Tracer.addAnnotation(String key, String value)

Tracer.addAnnotation(String key, String value, long timestamp)

返回值：void

为该次调用添加KV的信息对，mtrace会根据KV对在ES建立索引（不会进行分词），便于通过KV对在mtrace web平台查询指定的调用链路。

所以该KV对需要尽量简洁，比如传入userId： 123456，orderId： 11111这样的信息，之后就可以查询指定userId，orderId的相应链路了。

timestamp 不传入的话，会使用调用该接口的当前时间

注意：尽量不要传入大量的log数据，因为mtrace上报会对数据进行压缩，如果数据量太大，会对CPU有一定的消耗!!!

Tracer.info(String key, String value)

Tracer.info(String key, String value, long timestamp)

返回值：void

与addAnnotation接口类似，但是只会在调用链路上展示KV对，并不会建立索引。

所以该KV对需要尽量穿入一些有意义且又不用做索引的信息。

注意：尽量不要传入大量的log数据，因为mtrace上报会对数据进行压缩，如果数据量太大，会对CPU有一定的消耗!!!

3. 上下文接口

- i
1. 业务方请勿使用 MT、MTrace 做 key 的前缀，避免命名冲突。

2. 参数请勿传 null。

3. < 1.1.21 是版本对 context 数目和大小有限制：最多 50 个 KV，key、value长度不超过 256

Tracer.putContext(String key, String value)

Tracer.getContext(String key)

Tracer.clearContext(String key)

设置 forever context。在整条链路都支持上下文功能的前提下，永久上下文一旦创建，便会和整个调用链共享生命周期。

上下文的 key value 长度，及个数均有限制。具体根据版本有所不同，但不建议在上下文中保存过于复杂的信息。

Tracer.putOneStepContext(String key, String value)

Tracer.getOneStepContext(String key)

Tracer.clearOneStepContext(String key)

与上文的 forever context 区别在于：

- 1、此处设置的上下文信息仅仅会传递给调用链上的下一个环节。
- 2、get 方法获取到的是从上一个环节得到的参数，put 方法设置的是要传递给下一个环节的参数，二者操作的不是一个数据结构。

4. 埋点状态变更接口

Tracer.clientSend(TraceParam param)

返回值：Span

发起异步调用前埋点，表示埋点开始，异步接口不会把Span信息存入ThreadLocal

Tracer.clientRecv()

Tracer.clientRecvAsync(Span span)

返回值：Span

结束异步调用时埋点，表示埋点结束，异步场景需要手动将开始时生成的 Span 当作参数传递进来

Tracer.serverRecv(TraceParam param)

返回值：Span

服务端接收请求时调用，表示服务端埋点开始

Tracer.serverSend(Span span)

返回值：Span

服务端返回请求时调用，表示服务端埋点结束

5. 其它接口

Tracer.getCell()

Tracer.setCell(String cell)

Tracer.isGray()

设置/获取的 SET 名称，以及判断 SET 是否为灰度。

具体获取 SET 的逻辑如下：

- 1 上游流量中有 set 信息，以请求流量中的 set 信息为准
- 2 上游中无 set 信息，以本服务器机器上的 set 信息为准
- 3 上游中无 set 信息，本地无 set 信息（环境不支持，或者当前线程非请求处理线程，或者不在埋点中），返回null

注意：如果业务侧自定义埋点（非mtthrift，非pigeon，非mtrace http filter），但是协议不支持解析流量中的上下文信息，会导致获取的set信息为本地set信息

Tracer.getSwimlane()

Tracer.setSwimlane(String swimlane)

1.1.12 及后续版本生效。用于获取/设置泳道信息。

具体获取泳道信息的逻辑参见：[Mtrace 支持泳道2.0](#)

Tracer.enterSandbox()

Tracer.enterSandbox(String sandboxName)

1.3.0及后续版本生效。用于在子线程中获取新span，以解决浅拷贝问题。

PS:

1. 请务必在TraceRunnable、TraceCallable或mtrace封装的线程池中使用
2. 使用该接口将导致拓扑关系断开。

Tracer.exitSandbox()

1.3.0及后续版本生效。 与enterSandbox配合使用，在子线程处理完毕时结束沙盒中的span，并上报trace。

PS: 使用该接口将导致拓扑关系断开。

Tracer.watch(String key, ChangeListener changeLitener)

1.3.0及后续版本生效。 监听foreverContext中某个key的变化，变化包含ADD、DELETE、MODIFY。详见**ChangeListener**。

6. 手动强制设置 appKey

Tracer.setGlobalAppKeyForce(String appKey)

该方法设置的 appKey 优先级最高，高于从 app.properties 里读取的 appKey。

三、跨线程埋点方法

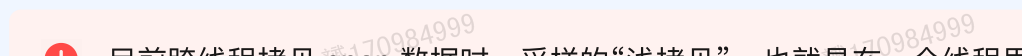
1. 线程封装

Mtrace 服务内部上下文传递依赖了ThreadLocal，但是涉及到跨线程的问题时，就会出现上下文传递断掉的问题。针对此问题对java线程库和线程池进行了封装，实现跨线程的上下文传递功能。

mtrace 上下文信息的传递是通过 ThreadLocal 实现的，所以当请求切换线程的时候，上下文信息在新的线程里会丢失。



这样每个线程就都能获取到完整的上下文信息，也能正确地生成 spanId。



之前有业务线遇到过，并行处理业务时，一个线程里删除了 set 信息，导致另一个线程里的 set 信息也被删除了，故在此特别说明。

TraceRunnable

代码块

java ▾ 

```
public Runnable getRunnable()
```

```
public Runnable getRunnable()
```

获取封装前的Runnable

```
public static TraceRunnable get(Runnable runnable)
```

代码块

```
1 TestRunnable testRunnable = new TestRunnable();
2 Thread t1 = new Thread(TraceRunnable.get(testRunnable));
```

获取封装后的Runnable 对象

```
public static List<TraceRunnable> gets(Collection<? extends Runnable> tasks)
```

获取封装后的Runnable 对象列表

代码块

```
1 public class TraceRunnableTest {
2     private static Logger logger = LoggerFactory.getLogger(TraceRunnableTest.class);
3
4     /**
5      * {@link TraceRunnable#TraceRunnable(Runnable)}
6      */
7     @Test
8     public void testNew() {
9         Span span = Tracer.serverRecv(new TraceParam(TraceRunnableTest.class.getName()));
10        Tracer.setTest(true);
11        TestRunnable testRunnable = new TestRunnable();
12        // 封装原有的Runnable
13        Thread t1 = new Thread(new TraceRunnable(testRunnable));
14        t1.start();
15        logger.info(span.toString());
16        Tracer.serverSend();
17    }
18
19    /**
20     * {@link TraceRunnable#get(Runnable)}
21     */
22    @Test
23    public void testGet() {
24        Span span = Tracer.serverRecv(new TraceParam(TraceRunnableTest.class.getName()));
25        Tracer.setTest(true);
26        TestRunnable testRunnable = new TestRunnable();
27        // 封装原有的Runnable
28        Thread t1 = new Thread(TraceRunnable.get(testRunnable));
29        t1.start();
30        logger.info(span.toString());
31        Tracer.serverSend();
32    }
}
```

```
33 }
34
35 class TestRunnable implements Runnable {
36
37     @Override
38     public void run() {
39         Assert.assertEquals(true, Tracer.isTest());
40     }
41 }
```

TraceCallable

public TraceCallable(Callable<V> callable)

^ 代码块 java

```
1 Callable<Span> callable = new TestCallable();
2 FutureTask<Span> task = new FutureTask<Span>(new TraceCallable<Span>(callable));
```

创建封装后的Callable 对象

public Callable<V> getCallable()

获取原始的Callable 对象

public static <T> TraceCallable<T> get(Callable<T> callable)

^ 代码块 java

```
1 Callable<Span> callable = new TestCallable();
2 FutureTask<Span> task = new FutureTask<Span>(TraceCallable.get(callable));
```

获取封装后的Callable 对象

public static <T> List<TraceCallable<T>> gets(Collection<? extends Callable<T>> tasks)

获取封装后的Callable 对象列表

^ 代码块 java

```
1 public class TraceCallableTest {
2     private static Logger logger = LoggerFactory.getLogger(TraceCallableTest.class);
3
4     /**
5      * {@link TraceCallable#TraceCallable(Callable)}
6      * @throws ExecutionException
7      * @throws InterruptedException
8      */
9     @Test
10    public void testNew() throws ExecutionException, InterruptedException {
```

```
Span span = Tracer.serverRecv(new TraceParam(TraceCallableTest.class.getName()));
Tracer.setTest(true);
Callable<Span> callable = new TestCallable();
// 封装原始的callable new TraceCallable<Span>(callable)
FutureTask<Span> task = new FutureTask<Span>(new TraceCallable<Span>(callable));
Thread t = new Thread(task);
t.start();
Span futureSpan = task.get();
Assert.assertEquals(futureSpan, span);
Tracer.serverSend();
}

/**
 * {@link TraceCallable#get(Callable)}
 * @throws ExecutionException
 * @throws InterruptedException
 */
@Test
public void testGet() throws ExecutionException, InterruptedException {
    Span span = Tracer.serverRecv(new TraceParam(TraceCallableTest.class.getName()));
    Tracer.setTest(true);
    Callable<Span> callable = new TestCallable();
    // 封装原始的callable TraceCallable.get(callable)
    FutureTask<Span> task = new FutureTask<Span>(TraceCallable.get(callable));
    Thread t = new Thread(task);
    t.start();
    Span futureSpan = task.get();
    Assert.assertEquals(futureSpan, span);
    Tracer.serverSend();
}
}

class TestCallable implements Callable<Span> {
    @Override
    public Span call() throws Exception {
        Assert.assertEquals(true, Tracer.isTest());
        return Tracer.getServerSpan();
    }
}
```

2. 线程池封装

为了保证跨线程传输 trace 信息的能力，mtrace 对 java 原生的线程池做了封装，具体对应关系为 原生线程池名字 + TraceWrapper，请根据需要取用。

注意：使用 Hystrix 的看 Wiki：[Hystrix：添加 Mtrace 埋点信息](#)。

⚠ 目前跨线程拷贝 span 数据时，采样的“浅拷贝”，也就是在一个线程里修改了 context 信息后，另一个线程也能看到。

之前有业务线遇到过，并行处理业务时，一个线程里删除了 set 信息，导致另一个线程里的 set 信息也被删除了，故在此特别说明。

Date: 2019-03-26

ExecutorTraceWrapper

public ExecutorTraceWrapper(Executor executor);

^ 代码块 java ▾

```
1 Executor executor = new ExecutorTraceWrapper(Executors.newSingleThreadExecutor());
```

ExecutorServiceTraceWrapper

public ExecutorServiceTraceWrapper(ExecutorService executorService);

^ 代码块 java ▾

```
1 Executor executor = new ExecutorServiceTraceWrapper(Executors.newSingleThreadExecutor());
```

ScheduledExecutorServiceTraceWrapper

public ScheduledExecutorServiceTraceWrapper(ScheduledExecutorService scheduledExecutorService);

^ 代码块 java ▾

```
1 Executor executor = new ScheduledExecutorServiceTraceWrapper(Executors.newSingleThreadScheduledExecutor());
```

TraceExecutors

public static Executor getTraceExecutor(Executor executor);

^ 代码块 java ▾

```
1 Executor executor = TraceExecutors.getTraceExecutor(Executors.newSingleThreadExecutor());
```

public static ExecutorService getTraceExecutorService(ExecutorService executorService);

^ 代码块 java ▾

```
1 Executor executor = TraceExecutors.getTraceExecutorService(Executors.newSingleThreadExecutor());
```

public static ScheduledExecutorService getTraceScheduledExecutorService(ScheduledExecutorService scheduledExecutorService);

^ 代码块 java ▾

```
1 Executor executor = TraceExecutors.getTraceScheduledExecutorService(Executors.newSingleThreadScheduledExecutor());
```

3. 线程池封装 (javaagent 方式)

设计详情见 [MTrace Agent 设计文档](#)。

❗

目前跨线程拷贝 span 数据时，采样的“浅拷贝”，也就是在一个线程里修改了 context 信息后，另一个线程也能看到。

之前有业务线遇到过，并行处理业务时，一个线程里删除了 set 信息，导致另一个线程里的 set 信息也被删除了，故在此特别说明。

Date: 2019-03-26

使用 javaagent 支持的场景如下：

- java.util.concurrent.ThreadPoolExecutor (从 1.1.15 版本开始)
- java.util.concurrent.ScheduledThreadPoolExecutor (从 1.1.15 版本开始)
- java.util.concurrent.ForkJoinPool & Java8 Stream (从 1.1.19 版本开始)

XFrame开发框架

如果使用 XFrame 可以参考 <https://km.sankuai.com/custom/onecloud/page/152184974#id-内置参数> 进行发布项配置

MDP开发框架

见 <https://docs.sankuai.com/dp/hbar/mdp-docs/master/trace/>

Tomcat war包部署

直接使用这种方式时，需要在 java 的启动参数添加 **-Xbootclasspath/a:mtrace-agent-xxx.jar具体路径 -javaagent:mtrace-agent-xxx.jar具体路径**

最好不要直接指定mtrace-agent的版本，避免jar包升级时找不到包，可以按如下配置例子，写在run.sh脚本里：

代码块Plant Text

```
1 cd $WEB_ROOT
2 ...
3 # 如果没有 WEB-INF/lib/ 目录，需要改成其它路径，比如 target/lib
4 MTRACE_AGENT=`ls WEB-INF/lib/ | grep mtrace-agent | grep -v "mtrace-agent-runtime"`
5 MTRACE_AGENT_PATH="WEB-INF/lib/${MTRACE_AGENT}"
6 JAVA_AGENT="-Xbootclasspath/a:${MTRACE_AGENT_PATH} -javaagent:${MTRACE_AGENT_PATH}"
7 ...
8 EXEC_JAVA="$EXEC $JAVA_CMD $JAVA_AGENT $JVM_ARGS $JVM_SIZE $JVM_HEAP $JVM_JIT $JVM_GC"\
9 ...
```

启动程序后，即可实现跨线程池传递参数

Springboot Jar包部署

- 1、首先在manifest.yamll 中 进行配置：

^ 参考manifest.yamlYAML

```
1
2 build:
3   ...
4   files:
5     - ./
6     - ../check.sh
7     - ../run.sh
8   ...
9   autodeploy:
10    targetDir: /opt/meituan/raptor-iot-api
11    run: sh run.sh
12    check: sh check.sh
13
14    checkRetry: 40
15    checkInterval: 5s
16
```

2、在run.sh 脚本中解压Fat_Jar 中的指定mtrace-agent-*.jar包 (需要在pom.xml中依赖，具体请参考接入文档)

^ run.shShell

```
1 # TARGET_DIR=/Users/ganghuanliu/IdeaProjects/arch/raptor-dashboard/dashboard/target
2 # FAT_JAR=dashboard-1.0-SNAPSHOT
3
4 cd $TARGET_DIR      #autodeploy中指定的targetDir
5
6 MTRACE_AGENT="mtrace_agent.jar"
7 agent_jar=`unzip -l $FAT_JAR.jar |grep  -E "mtrace-agent-*.jar"|grep -v "mtrace-agent-runtime-*.jar"|awk '{print $4}'`
8 echo mtrace agent is $agent_jar
9 unzip -p  $FAT_JAR.jar $agent_jar > $MTRACE_AGENT
10
11 if [ ! -f mtrace_agent.jar ]; then
12   echo "依赖的 mtrace_agent jar不存在"
13   #   exit 0
14 fi
15
16 JAVA_AGENT="-Xbootclasspath/a:${MTRACE_AGENT} -javaagent:${MTRACE_AGENT}"
17 java $JAVA_AGENT -jar $FAT_JAR.jar
18 ## macos的zsh下测试时请使用:
19 ## $(echo "java $JAVA_AGENT -jar $FAT_JAR.jar")
```

运行时动态attach mtrace agent

使用说明：参考 参数配置 设置mtrace.agent.attach=true，将会自动在运行时加载mtrace-agent，将会看到如下的日志：

 jdk11以上需要在启动参数中增加 -Djdk.attach.allowAttachSelf=true

该日志使用java.util.logging输出，可能输出到标准输出，可到 /opt/logs/logs/plus_stdout/hulk_start_log.xxxxxx查看

代码块

Shell

```
1 [04-12 10:19:13] [INFO] [http-nio-8080-exec-1] [sg.MtraceConfigByMcc] auto attach mtrace agent...
2 [04-12 10:19:13] [INFO] [http-nio-8080-exec-1] [agentload.AgentLoader] copy agent.jar from mtrace-agent-runtime.jar.resource t
/tmp/mtrace-agent-runtime12128248720293940733.jar
3 [04-12 10:19:13] [INFO] [Attach Listener] [runtime.AgentMain] clazz need transform is [ForkJoinPool, ThreadPool, ScheduledPool
4 [04-12 10:19:13] [WARN] [Attach Listener] [runtime.AgentTool] ForkJoinTask.class has been loaded. ForkJoinPool instrumentation
failed.
5 [04-12 10:19:13] [INFO] [Attach Listener] [runtime.AgentTool] not transform ForkJoinPool because ForkJoinTask has been loaded.
6 [04-12 10:19:13] [INFO] [Attach Listener] [transformer.AbstractTransformer] Transforming class
java/util/concurrent/ThreadPoolExecutor because of retransforming
7 [04-12 10:19:13] [INFO] [Attach Listener] [transformer.AbstractTransformer] Transforming class
java/util/concurrent/ScheduledThreadPoolExecutor because of retransforming
8 [04-12 10:19:13] [INFO] [http-nio-8080-exec-1] [agentload.AgentLoader] delete mtrace-agent-runtime12128248720293940733.jar
successfully
9 [04-12 10:19:13] [INFO] [http-nio-8080-exec-1] [agentload.AgentLoader] 当前的类加载器: TomcatEmbeddedWebappClassLoader
context: ROOT
11 delegate: true
12 -----> Parent Classloader:
13 org.springframework.boot.loader.LaunchedURLClassLoader@457e2f02
14
15 [04-12 10:19:13] [INFO] [http-nio-8080-exec-1] [agentload.AgentLoader] mtrace agent load success
```

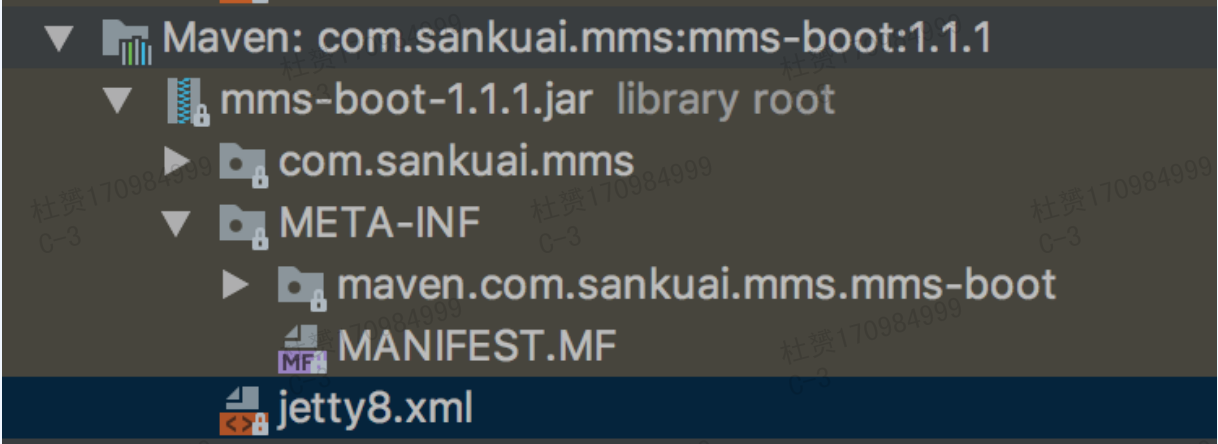
Jetty 特殊配置

当使用 jetty 启动时，需要设置 parentLoaderPriority 为 true。否则，webappclassloader 和 bootstrapclassloader 会同时加载 mtrace-agent.jar 里的类，造成 Trace 信息传递失败。详细分析见：[血泪的 Jetty ClassLoader](#)。

当使用公司组件 mms-boot 启动应用时，可以在 jetty.xml 里配置 parentLoaderPriority 属性。如果不使用 jetty.xml，直接调用方法即可。参见<https://www.eclipse.org/jetty/documentation/current/jetty-classloading.html>

注意：jetty.xml 放在 resources/jetty 文件夹下。(重点是 parentLoaderPriority 必须为 true，其他参数和此处目的无关)

 部分版本的 mms-boot 里 parentLoaderPriority 默认就是 true，也就不用在项目里新增 jetty.xml 配置了。具体默认配置，看 jar 包里的 jetty8.xml。



```
jetty.xml
1  <?xml version="1.0"?>
2  <!DOCTYPE Configure PUBLIC "-//Mort Bay Consulting//DTD Configure//EN" "http://jetty.eclipse.org/configure.dtd">
3  <Configure id="Server" class="org.eclipse.jetty.server.Server">
4      <Set name="handler">
5          <New id="Handlers" class="org.eclipse.jetty.server.handler.HandlerCollection">
6              <Set name="handlers">
7                  <Array type="org.eclipse.jetty.server.Handler">
8                      <Item>
9                          <New class="org.eclipse.jetty.webapp.WebAppContext">
10                             <Set name="tempDirectory">
11                                 <SystemProperty name="java.io.tmpdir" default="/tmp" />
12                             </Set>
13                             <Set name="resourceBase">
14                                 <SystemProperty name="jetty.webroot" />
15                             </Set>
16                             <Set name="contextPath">
17                                 <SystemProperty name="jetty.context" />
18                             </Set>
19                             <Set name="parentLoaderPriority">true</Set>
20                             <!-- <Set name="defaultsDescriptor"><SystemProperty name="jetty.home"/>/etc/webdefault.xml</Set> -->
21                             <Set name="configurationClasses">
22                                 <Ref id="plusConfig" />
23                             </Set>
24                         </New>
25                     </Item>
26                     <Item>
27                         <New class="org.eclipse.jetty.server.handler.RequestLogHandler">
28                             <Set name="requestLog">
29                                 <New id="RequestLogImpl" class="org.eclipse.jetty.server.AsyncNCSARequestLog">
30                                     <Set name="filename"><SystemProperty name="jetty.logs" default="./logs" /></SystemProperty
31                                     name="jetty.appkey" default="jetty" />.request.log.yyyy_mm_dd</Set>
32                                     <Set name="filenameDateFormat">yyyy-MM-dd</Set>
33                                     <Set name="retainDays">90</Set>
34                                     <Set name="append">true</Set>
35                                     <Set name="extended">true</Set>
```

```
35         <Set name="logCookies">false</Set>
36         <Set name="preferProxiedForAddress">true</Set>
37         <Set name="LogTimeZone">GMT+8</Set>
38         <Set name="logLatency">true</Set>
39         <Set name="logServer">true</Set>
40     </New>
41 </Set>
42 </New>
43 </Item>
44 </Array>
45 </Set>
46 </New>
47 </Set>
48 </Configure>
```

适用场景

- 1. 不想用 Wrapper 方式手动改代码的。
- 2. 使用了 jdk8 并行 stream、CompletableFuture 等新特性的。

4. 跨线程传递 ThreadLocal 变量

在某些业务场景下，业务想要实现跨线程传递某个 ThreadLocal 变量，但又不想把数据放在 MTrace 的 Context 里，该怎么办呢？

可以使用 TransmissibleThreadLocal 类来存储数据，即可实现跨线程传输 ThreadLocal 变量。需要注意的是，和跨线程传递 Trace 数据一样，都需要在代码中使用 Wrapper 或配置 javaagent 参数。

❗ 该特性需要 mtrace-agent 1.1.22 及以上版本。
主线程里的 ThreadLocal 变量还是需要业务自己释放的。

代码块 Java

```
1 import com.meituan.mtrace.context.TransmissibleThreadLocal;
2 public static ThreadLocal<String> var = new TransmissibleThreadLocal<>();
```

5. Springboot @Async透传mtrace信息

见 [Spring @Async底层线程池使用mtrace线程池封装](#)

四、AOP 方法埋点使用教程

五、参数配置

1. 配置项说明

- ❗ 1. mcc 配置需要 mtrace 版本升级至 1.1.16.2 以上。

- 2. 尽量保持默认值，不要轻易更改参数值
- 3. 采样率/QPS 配置需要在链路的根节点所在系统设置才有效。
- 4. 系统整体 QPS 小于 2000 才可设置全量采集。
- 5. 推荐升级最新版本，配置 mtrace.sample.rate 参数。

参数获取优先级：**app.properties(classpath:/META-INF/app.properties) > mcc (octo mcc 动态配置项)**

参数名	说明	默认值	备注
mtrace.upload.sgagent	是否收集上下文数据	true	
mtrace.upload.single.span.max.num	上报 MTrace 数据一批最多数据条数，一般情况下不用调整，仅当用户自定义信息（Tracer.info、addAnnotation）比较多，导致数据量较大时，可能超过mafka 上报数据限制时，需要调整	4096	mtrace/mtrace-api/mtrace-agent/mtrace-http 版本 >= 1.1.24
mtrace.upload.max.interval	数据上报最大时间间隔	8192	mtrace/mtrace-api/mtrace-agent/mtrace-http 版本 >= 1.1.24
mtrace.slowquery.threshold	慢查询参数，大于这个阈值的请求会被存储，可以在mtrace web平台上查询	500	时间单位 ms
mtrace.compress.type	上报数据的压缩方式	snappy	可选参数 gzip / snappy (M1芯片的mac请使用gzip)
mtrace.agent.attach	是否自动加载mtrace-agent做自动跨线程封装	false	mtrace/mtrace-agent/mtrace-agent-runtime 版本 >= 1.3.1 如果已经使用mdp的跨线程封装，无需使用该项配置
mtrace.spanname.limit	spanName个数限制	200	mtrace 版本 >= 1.3.1
mtrace.id.version	traceld生成方案	v2	v2:强随机数生成方案 v3:伪随机数生成方案，可能存在id碰撞的情况

以下参数不建议设置，会影响稳定性，如需请tt咨询

✕ 点击展开内容

2. 关于强制上报的基础组件配置的说明（http、mtthrif**t**、zebra）

mtrace.upload.infraName 这个配置中的基础组件，由于涉及到 octo 等其它系统的监控、统计功能，所以采用的是全量上报，由 mtrace 采样存储的方式。将此配置配成空字符串，可以使 mtthrif**t** 等组件也按照采样逻辑进行上报（不推荐）。

这个地方的配置如果要全量采集，需要在代码中调用 Tracer.setDebug(true)（需要考虑 QPS 问题）。

3. 关于采样配置的说明

未包含在 `mtrace.upload.infraName` 中的链路数据，会按照采样策略上报数据。配置 `mtrace.sample.rate` 和 `mtrace.sample.ratio` 会影响正常链路的采样，异常链路会走强制上报逻辑。

以 `mtrace.slowquery.threshold` 为例，即使你配置了 `ratio = rate = 0`，只要遇到了慢请求，还是会被采样上报。

如果想要修改采样率（请注意性能损耗），建议 `ratio` 和 `rate` 一起配置，避免版本升级带来的配置失效问题。

4. 配置生效验证

使用 `mcc` 修改配置或初次读入本地文件后，`mtrace` 会打出一条 `INFO` 日志，展示当前的配置内容。

以 1.1.21 版本修改采样率为例：在修改 `mcc` 配置后，会收到一条变更提醒：

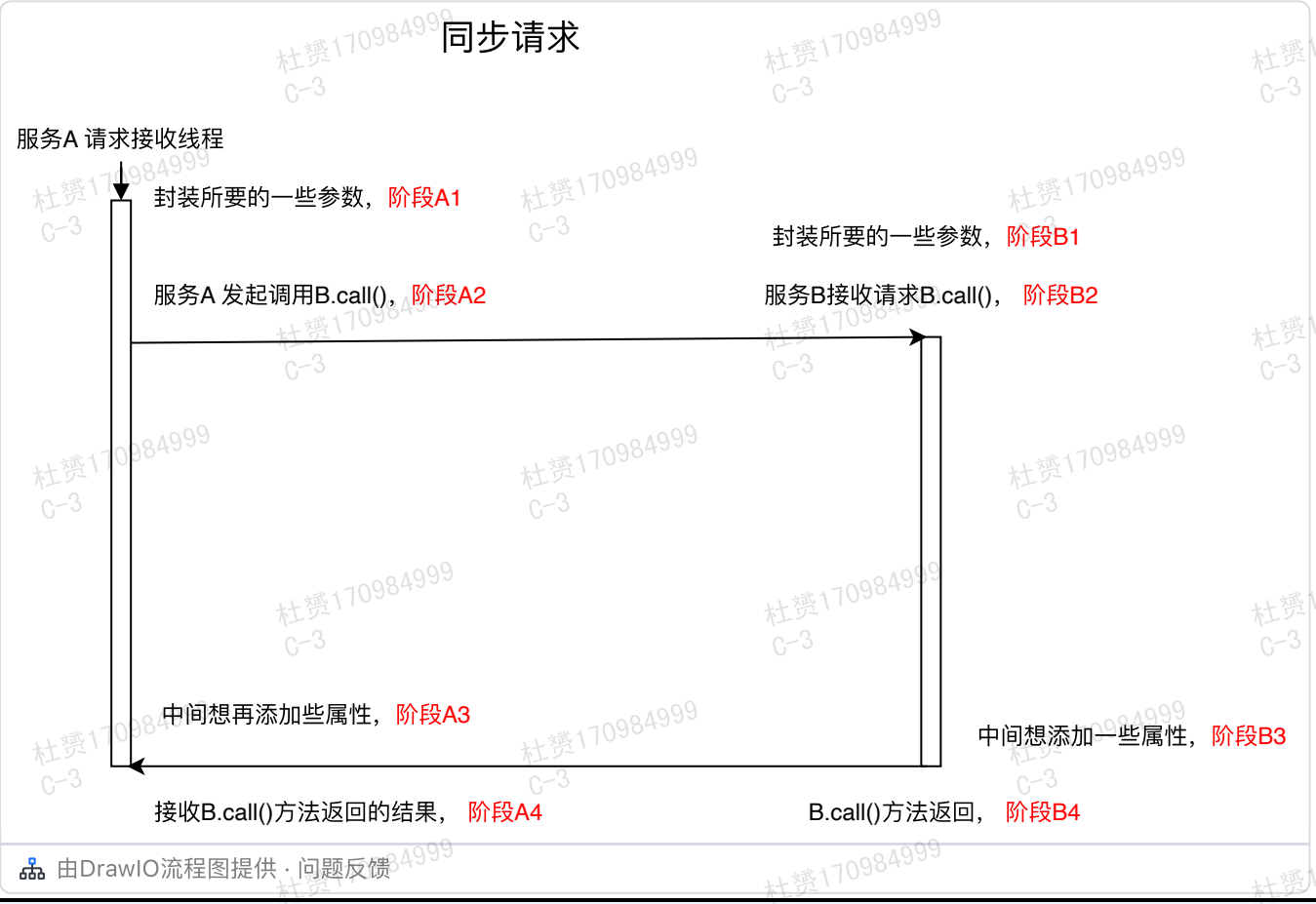
```
2019-03-14 18:17:07
MCC配置变更提醒(线上prod)
Appkey:com.sankuai.waimai.sc.saasmif
env:prod
操作人:任松良
【新增配置】
#每分钟采样个数
mtrace.sample.rate = 10000
```

确保 `com.meituan.mtrace` 包的 `INFO` 日志能够输出后，可以在日志中找到如下内容：

```
2019-03-14 18:17:32.188 - - [INFO] mtrace-config MtraceConfigByMcc mtrace properties mtrace properties{appkey:com.sankuai.waimai.sc.saasmif, mtrace.slowquery.threshold:500, mtrace.compress.type:gzip, mtrace.upload.sgagent:true, mtrace.upload.infraName:[mtthrift, HTTP, zebra, MTDDL], RateSpanSampler{sampleCountPerMinute=10000.0}}
```

六、使用 demo

1. 同步请求流程



2. 服务A (client) 状态说明

阶段A1（非必需）

代码块

java

```
1  TraceParam param = new TraceParam("method");
2  param.setInfraName("infraName");
3  param.setRemoteAppKey("remote appkey");
4  param.setRemoteIp("127.0.0.1");
```

需要在埋点前构建一个数据结构存储相关信息的时候可以使用该变量，如果不需要可以不使用，在后面的埋点过程中装填。

TraceParam 属性	Desc	网络传递字段
traceld	server端需要把网络中传过来的traceld填进来	true
spanId	server端需要把网络中传过来的spanId填进来	true
spanName	类名 + “.” + 方法名	
remoteAppKey	远端的appkey	
remoteIp	远端的ip	
infraName	中间件名称 EG : mtthrift, cellar, zebra	
version	中间件版本 EG : xx.xx.xx	
packageSize	包大小，单位字节	
debug	server端需要把网络中传过来的debug字段填进来	true
foreverContext	server端需要把网络中传过来的foreverContext字段填进来	true
oneStepContext	server端需要把网络中传过来的oneStepContext字段填进来	true

阶段A2

代码块

java

```
1  // 以下是埋点的三种方式
2  Tracer.clientSend(param); // 方式1 参数是TraceParam
3  Tracer.clientSend("Class.method"); // 方式2 参数是 类名.方法名
4  Tracer.clientSend(); // 方式3 无参数
```

- Tracer.clientSend(param) 把TraceParam 当作参数传入埋点开始接口，会把TraceParam设置的参数装填到埋点上下文中
- Tracer.clientSend("Class.method") 埋点开始接口，会自己构建一个埋点上下文，名称是Class.method
- Tracer.clientSend() 埋点开始接口，会自己构建一个空的埋点上下文，如果后面到埋点结束也没有添加名称，则不会上报信息

阶段A3

^ 代码块

1 ITracer tracer = Tracer.getClientTracer(); // 获取当前埋点对应的tracer

2 tracer.setSize(100); // 设置接收到的请求包大小

3 tracer.setStatus(Tracer.STATUS.SUCCESS); // 设置请求的返回结果

java

1. 获取埋点上下文封装对象
2. 如果能获取到上下文信息，则设置请求包大小
3. 如果能获取到上下文信息，则设置请求返回状态

setName	设置请求接口名
setStatus	设置请求状态
setInfraName	设置中间件名称
setSize	设置请求包大小
setAsync	设置是否为异步调用
setSample	设置是否采样标识
putContext	设置传递上下文数据
putLocalOneStepContext	设置本地传递一级上下文数据
putRemoteOneStepContext	设置远程传递一级上下文数据
info	设置调用信息属性
addAnnotation	设置调用信息属性（会建立索引）

阶段A4

^ 代码块

1 Tracer.clientRecv();

java

请求埋点结束

3. 服务A (Client) 接入样例

^ 代码块

1 TraceParam param = new TraceParam("B.call()");

2 param.setInfraName("mtthrift");

java

```
3 param.setRemoteAppKey("com.meituan.service.B");
4 param.setRemoteIp("10.11.24.132");
5
6 Tracer.clientSend(param); // 埋点开始
7
8 ITracer tracer = Tracer.getClientTracer(); // 获取当前埋点对应的tracer
9 tracer.setSize(100);
10 try {
11     Response res = B.call(...); // 中间件调用
12     tracer.setStatus(Tracer.STATUS.SUCCESS);
13 } catch (Exception e) {
14     tracer.setStatus(Tracer.STATUS.EXCEPTION);
15 } finally {
16     Tracer.clientRecv(); // 埋点结束
17 }
18
```

4. 服务B (server) 状态说明

阶段B1

代码块

```
1 TraceParam param = new TraceParam("method");
2 param.setInfraName("infraName");
3 param.setRemoteAppKey("remote appkey");
4 param.setRemoteIp("127.0.0.1");
5
6 /*
7  * 如果网络中传递过来了trace信息，需要装填网络传递过来的trace信息
8  */
9 param.setTraceId(traceId);
10 param.setSpanId(spanId);
11 param.setDebug(debug);
12 param.setForeverContext(foreverContext);
13 param.setOneStepContext(oneStepContext);
```

- traceId 请求唯一标识
- spanId 请求位置标识
- debug 是否采集存储
- foreverContext 自定义传递上下文
- oneStepContext 自定义传递一级的上下文

阶段B2

^ 代码块

1 // 以下是埋点的三种方式

2 Tracer.serverRecv(param); // 方式1 参数是TraceParam

3 Tracer.serverRecv("Class.method"); // 方式2 参数是 类名.方法名

4 Tracer.serverRecv(); // 方式3 无参数

阶段B3

^ 代码块

1 ITracer tracer = Tracer.getServerTracer(); // 获取当前埋点对应的tracer

2 tracer.setSize(123); // 设置接收到的请求包大小

3 tracer.setStatus(Tracer.STATUS.SUCCESS); // 设置请求的返回结果

4 tracer.info("serialize", "12", System.currentTimeMillis()); // 设置中间件序列化所耗费的时间, KV结构, 可以自行定义

阶段B4

^ 代码块

1 Tracer.serverSend();

5. 服务B (server) 接入样例

^ 代码块

1 TraceParam param = new TraceParam("B.call()");

2 param.setInfraName("mtthrif");

3 param.setRemoteAppKey("com.meituan.service.A");

4 param.setRemoteIp("127.0.0.1");

5

6 param.setTraceId(traceId);

7 param.setSpanId(spanId);

8 param.setDebug(debug);

9 param.setForeverContext(foreverContext);

10 param.setOneStepContext(oneStepContext);

11

12 // 请求接收处理线程

13 Tracer.serverRecv(param); // 埋点开始

14

15 ITracer tracer = Tracer.getServerTracer(); // 获取当前埋点对应的tracer

16 tracer.setSize(111);

17 tracer.info("serialize", "12", System.currentTimeMillis());

18 tracer.info("getConnection", "8", System.currentTimeMillis());

19 try {

20 // 业务端逻辑处理部分

```
21         tracer.setStatus(Tracer.STATUS.SUCCESS);
22     } catch (Exception e) {
23         tracer.setStatus(Tracer.STATUS.EXCEPTION);
24     } finally {
25         Tracer.serverSend(); // 埋点结束
26     }
27
```

6. 跨线程请求流程

1.5 使用实现了埋点功能的中间件

当 server 使用一个包含了埋点功能的中间件时，处理业务逻辑的代码通常是包含在 span 的生命周期之中的。

以 mtrace-http 为例，当你开始处理 request 时，mtrace-http 已经调用了 ServerRecv 方法，开始了 server span 的生命周期，而业务逻辑结束给 client 返回结果时，中间件又会调用 ServerSend 方法结束埋点过程。在这个过程中你可以获取上游传递的链路特征、压测标记、采集信息等埋点数据并做业务逻辑上的特殊处理。

如果是 client 使用中间件，通常中间件内部实现的是 ClientSend 和 ClientRecv 方法。

以 mtthrift 为例，在发起 rpc 调用时，mtthrift 会将当前环节的埋点信息 (traceId 等) 同时发送给下游，避免调用链断裂。

 mtthrift 等公司内部提供的中间件自动支持 MTrace，而 HTTP 是需要额外配置或改代码才可支持的，具体见子节点 wiki。

2 工作原理

2.1 串行任务模型

通常的服务框架设计中，同一个线程同一时间只会处理一个请求，因此可以使用 Thread Local 变量存储 client 及 server 的 Span 对象。仍旧以上文的 B 为例：

^ B 的服务线程逻辑

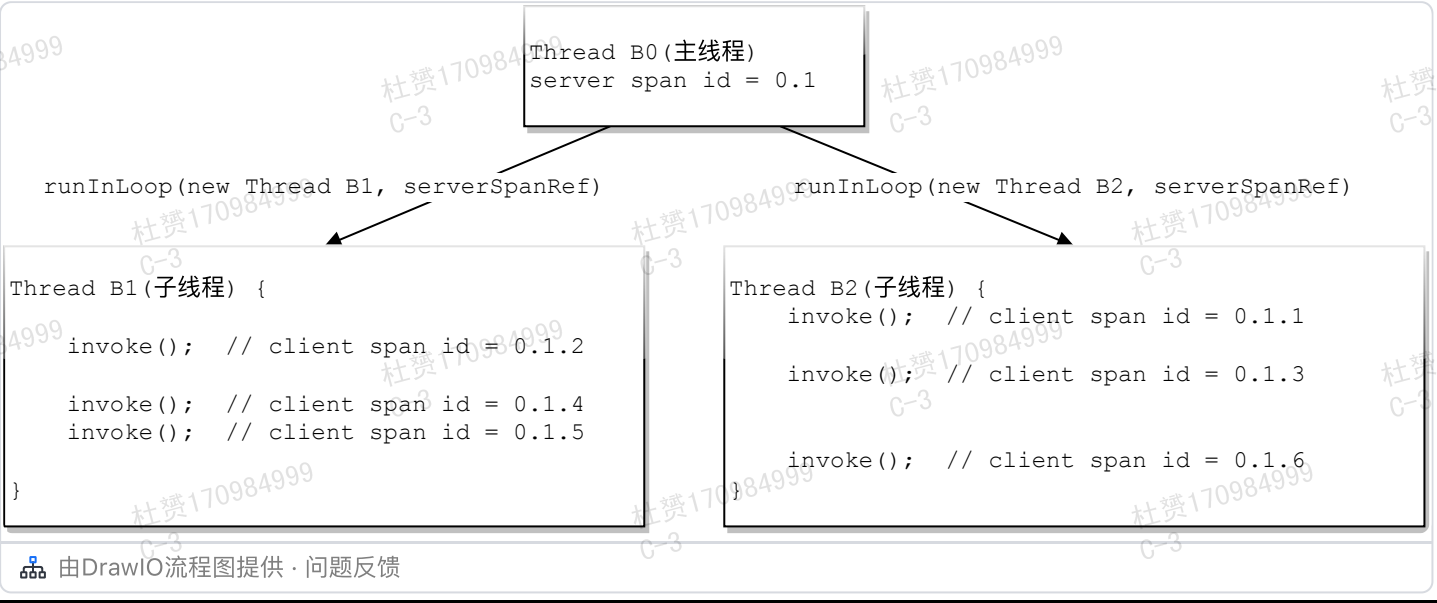
```
1  public void handle(Request req) {
2      Tracer.serverRecv(); // 生成 threadlocal server span
3      preProcess(req); // 请求预处理逻辑
4      callC(); // 发起 rpc 调用
5      callD(); // 发起 rpc 调用
6      postProcess(req); // 请求处理逻辑
7      Tracer.serverSend(); // 上报并清理 threadlocal 变量
8  }
9
10 public void callC() {
11     Tracer.clientSend(); // 生成 threadlocal client span 1
12     C.invoke(); // 发起 rpc 调用
13     Tracer.clientRecv(); // 清理 threadlocal client span 1
14 }
15
```

```
16 public void callD() {
17     Tracer.clientSend(); // 生成 threadlocal client span 2
18     D.invoke();          // 发起 rpc 调用
19     Tracer.clientRecv(); // 清理 threadlocal client span 2
20 }
```

2.2 并行任务模型

如果 B 在处理请求时并发地执行了多个 client span，则多个 client span 会共享 server span（需要使用 mtrace 封装的线程池），并按照启动顺序分别生成自己的 client span。

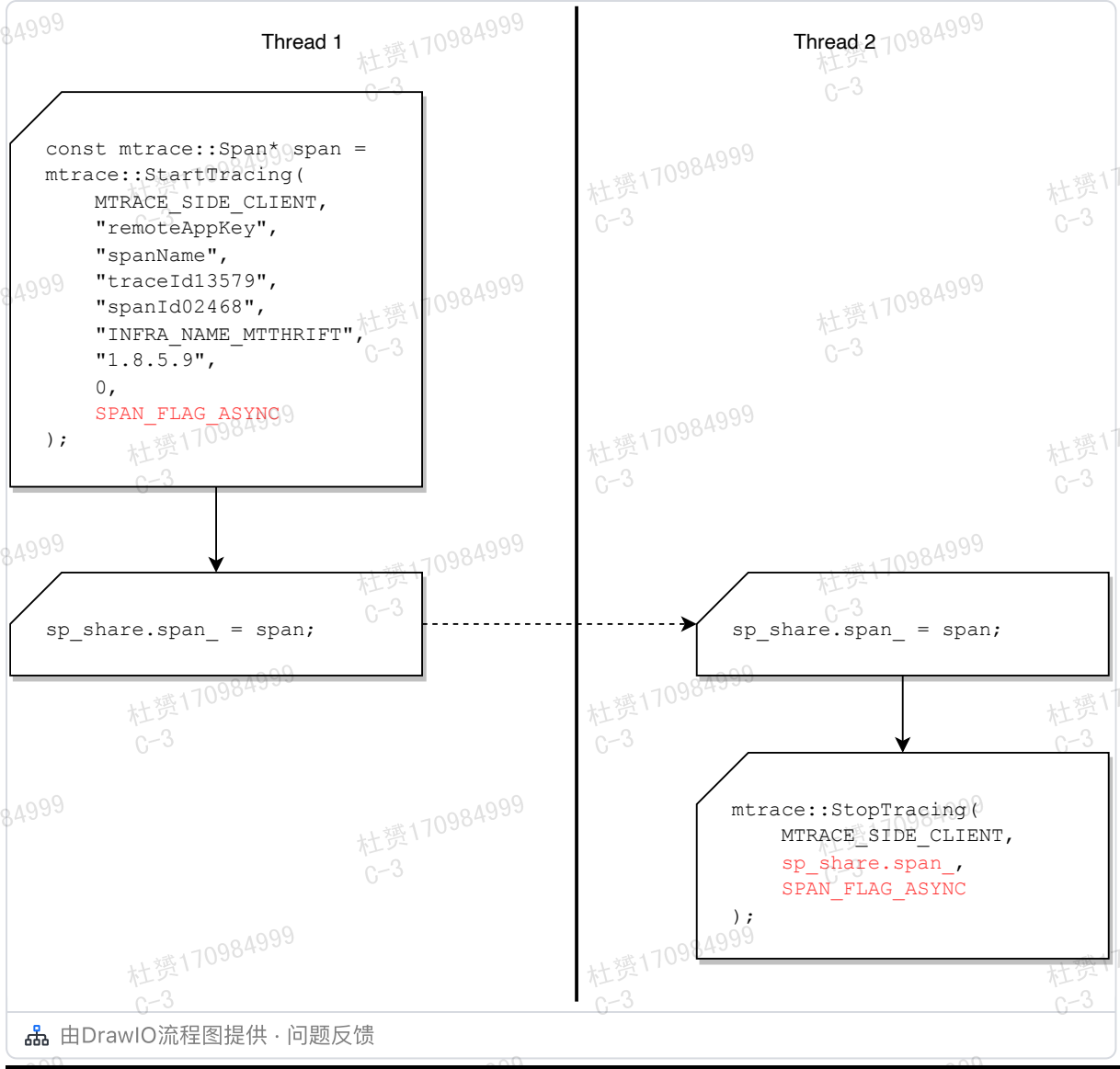
client span id 的赋值是根据 server span 里的计数器来的(atomicInt.incrementAndGet())，所以不用担心重复的问题。



2.3 异步模型

如果服务端使用异步+回调的方式处理请求，需要把 span 参数传递给回调线程，否则 span 信息会丢失。

具体的支持情况请询问你使用的中间件开发者。



3 数据采集

有了Trace Id和Span Id，我们只需要在上传log的时候带上这两个字段，就可以将一次请求串联起来。

比如上例中：

1. A接收到请求的时候生成traceld 123456和spanId 0,然后记录 A的服务名接口名，Ip,port 和接收请求的处理时间
2. 当A作为client端发送请求的时候将traceld 和spanId 埋入rpc请求调用的网络字节流中，并记录A调用B的时间信息，B的服务名，接口名等等信息
3. B接收请求的时候获取网络字节流中的traceld 和spanId，然后记录自己收到请求的时间和自己的服务名，接口名，Ip，port等信息
4. ...

由此可形成如下数据：

1. traceld 123456, spanId 0 , isClientSide false, appKey A, spanName methodA, startTime 100000000, duration 50, ip 127.0.0.1, port 8080 (由A记录)
2. traceld 123456, spanId 0.1 , isClientSide true , appKey B, spanName methodB, startTime 100000010, duration 20, ip 127.0.0.2, port 8081 (由A记录)
3. traceld 123456, spanId 0.1 , isClientSide false , appKey B, spanName methodB, startTime 100000012, duration 17, ip 127.0.0.2, port 8081 (由B记录)
4.

3.1 数据采集

因为如果每次rpc都上报log的话会对系统带来灾难性的带宽消耗，所以我们需要按一定的方式采样，为了能拼成相应的调用链路，我们采取在根节点服务采样（spanId为0的服务），之后的服务都按根节点的方式进行采样。

每个服务内部都有一个采样的队列，被采样的数据会放到采样队列中进行上报，当队列满的情况下多余数据会被 drop。



由于采用了采样、有损的采集方式，所以 mtrace 最终保存的数据必然是总数据的一个子集。

3.2 目前的数据上报/数据采样策略

目前一些中间件的链路信息会全量上报（比如 http、thrift、zebra），以满足其它平台对链路数据的需求（比如 octo 统计的指标数据），然后由 mtrace 负责统一采样存储。

其它情况则只会上报采样的数据，mtrace 系统不再进行二次采样，直接存储数据。

对调用链的采样存储策略包括按照概率采样、按照频率（次/分钟）采样等等。另外，为了保证能够定位问题，mtrace 在处理数据时还做了 [Mtrace 异常链路存储](#) 策

阿里鹰眼

概述

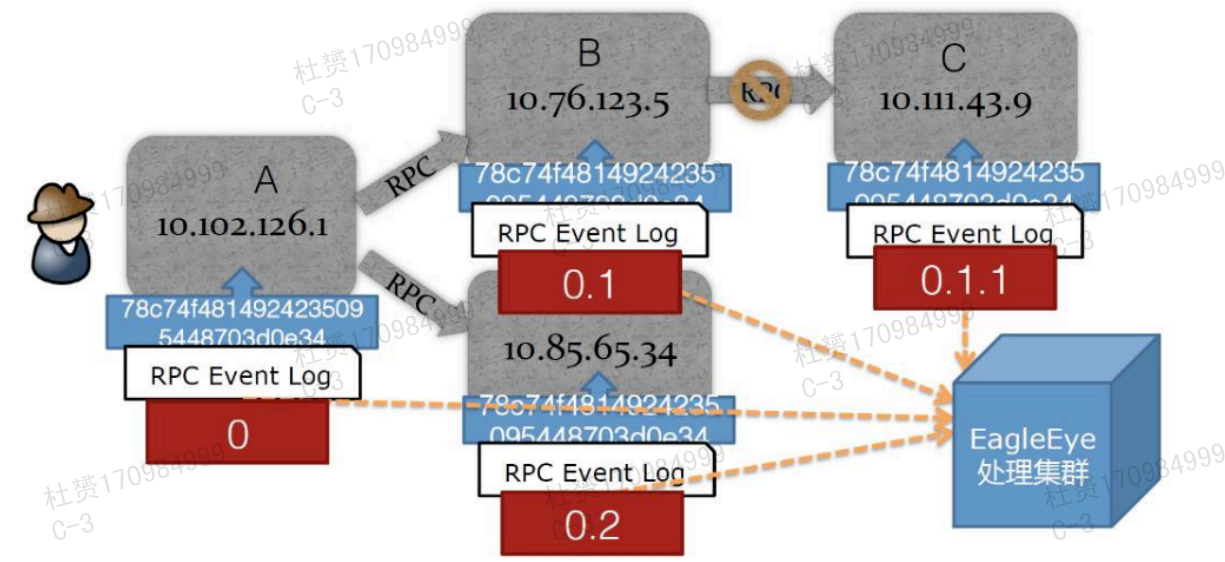
最新一代的阿里全链路监控系统鹰眼 3.0，同时将基础设施层、分布式应用层、业务逻辑层与客户端层进行了全链路跟踪；技术层面，鹰眼 3.0 日均处理万亿级别的分布式调用链数据，针对海量实时监控的痛点，对底层的流计算、多维时序指标与事件存储体系等进行了大量优化，同时引入了时序检测、根因分析、业务链路特征等技术，将问题发现与定位由被动转为主动。

基于微服务体系之下构建的业务系统存在的问题基本上分为四类：故障定位难、容量预估难、资源浪费多、链路梳理难。鹰眼的存在，就是为了解决这些问题。鹰眼是一个以链路跟踪技术为核心的监控系统，通过收集、存储、分析分布式系统中的调用事件数据，协助开发和故障运维人员进行故障诊断、容量预估、性能瓶颈定位、调用链路梳理。它的灵感是来自于 Google 的 Dapper。

基本原理

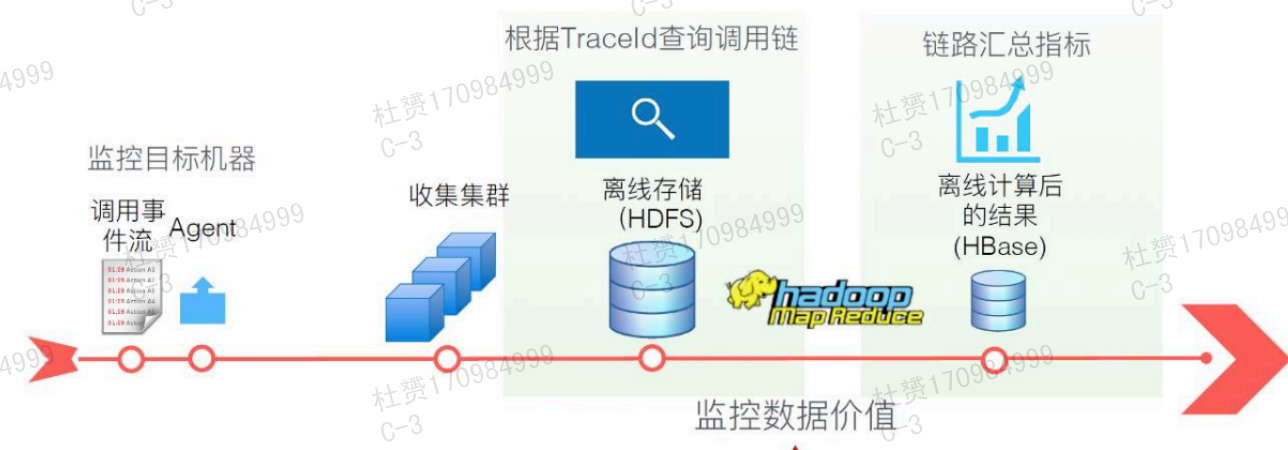
鹰眼的核心是链路跟踪技术，链路跟踪技术的基本原理是：使用 traceID 唯一标识一次调用链，使用 spanID 来标记计算节点在调用链中的位置。

一次分布式调用 TraceID + RPCID(SpanID)

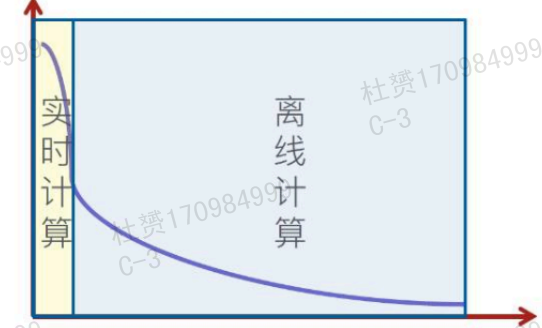


系统架构

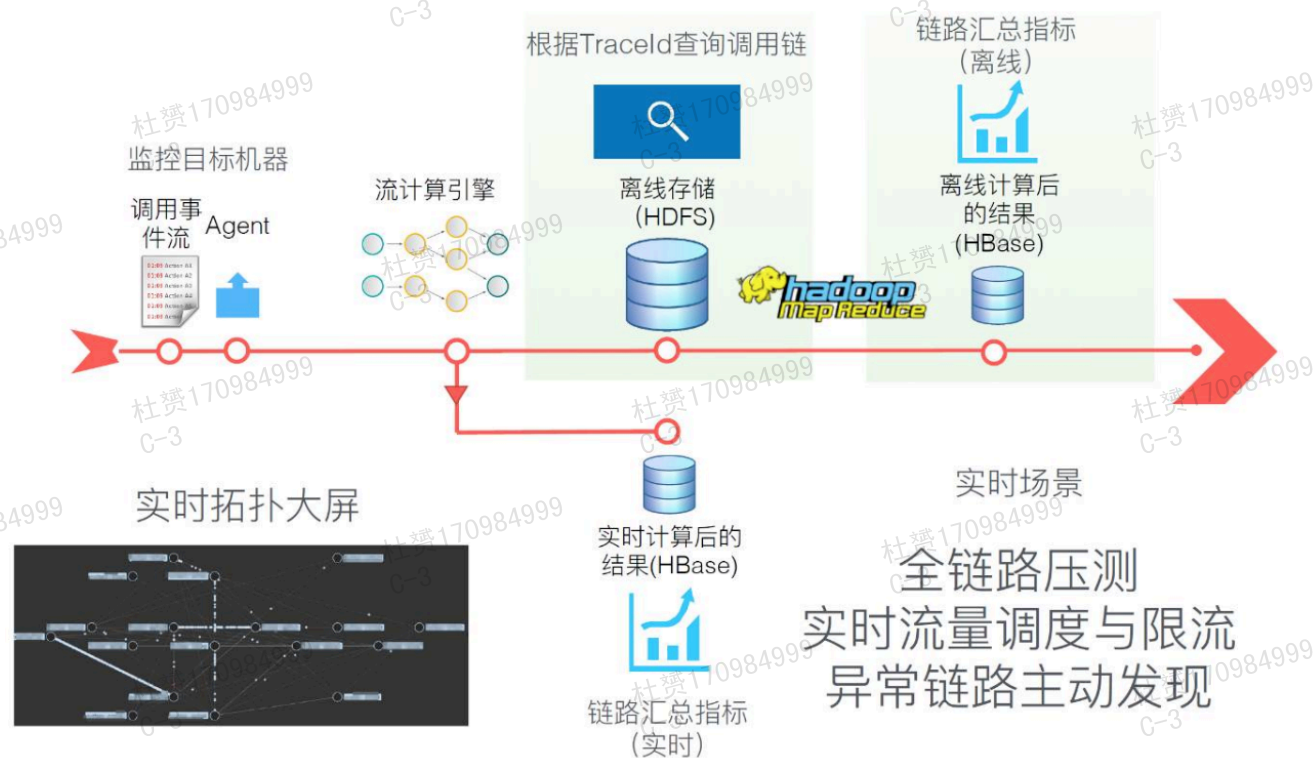
鹰眼整体架构 (2012)



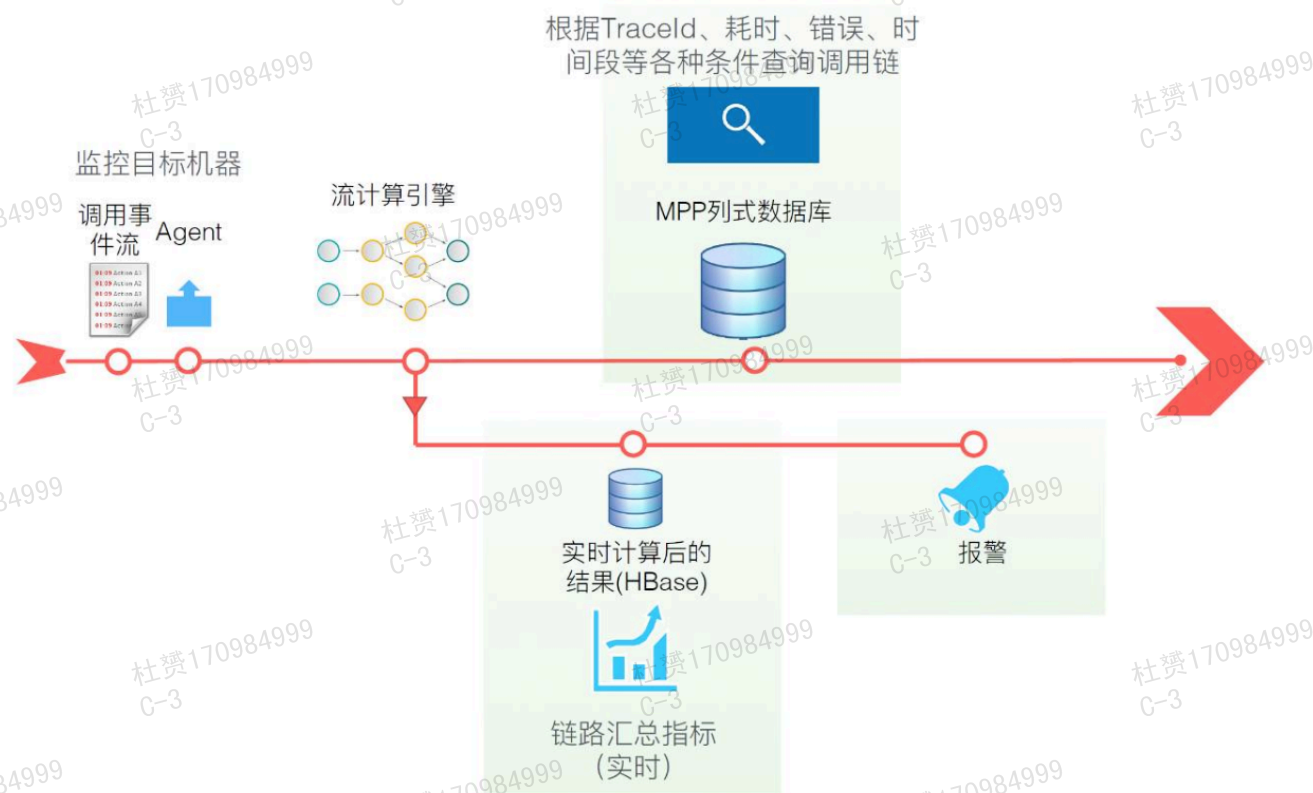
- ### 主要问题
- 1. 不够实时
 - 2. 不够轻量



鹰眼整体架构 – 实时化 (2014)



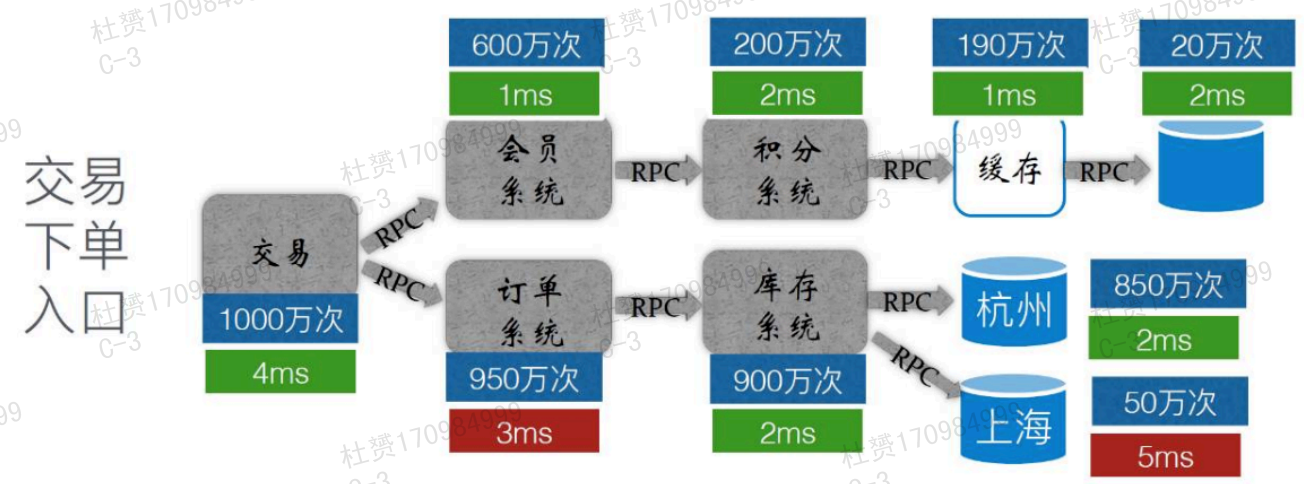
鹰眼整体架构 – 轻量化 (2016)



主要功能

发现瓶颈、容量预估

将万亿级别调用链数据进行聚合 (根据业务入口进行聚合)



价值:

- 1. 发现热点, 发现瓶颈, 发现“非法流量”。
- 2. 容量预估。(例: 按照双十一各入口流量预估值, 按照链路下游各系统的调用比例, 计算出双十一各子系统所需承担的流量。)

调用链的聚合结果 - 链路分析

下单链路

层次	名称	QPS	峰值QPS	调用比例	被调用比例	平均耗时	出错率	同机房	标记
0	http://	82.66	8700	1.0000	1.0	312ms	16.52%	0.00%	瓶颈
1		1.83	34450	0.0022	11.47	0ms	0.01%	0.00%	依赖压力点
1		439.22	22200	5.3138	5.45	0ms	0.86%	0.02%	
1		0.21	3680	0.0025	1.02	2ms	0.00%	0.00%	
2		0.21	3660	0.0025	1.02	0ms	0.00%	0.00%	
2		0.13	20	0.0016	1.0	1ms	0.00%	0.00%	易故障点
1		80.61	8480	0.9752	1.0	5ms	1.49%	1.56%	潜在易故障点
2		0.0	130	0.0000	1.0	10ms	0.00%	0.00%	
2		0.01	190	0.0001	2.13	10ms	0.00%	0.00%	
2		0.01	130	0.0001	2.27	0ms	0.00%	0.00%	
2		0.01	190	0.0001	2.13	0ms	0.00%	0.00%	
1		79.45	8440	0.9612	1.0	0ms	0.08%	0.00%	
1		0.85	60	0.0103	1.09	5ms	0.01%	0.01%	耗时瓶颈点
1		0.15	520	0.0018	1.0	107ms	34.29%	0.00%	
1		0.08	30	0.0010	1.0	2ms	0.00%	0.00%	
1		0.15	520	0.0018	1.0	0ms	0.00%	0.00%	

StreamLib 提升流计算性能

监控系统 - 流计算的挑战

- 计算过程不落地
有限内存 vs 海量数据
热点问题
- 解决方案
全量计算->流式增量计算（例：全量数据
计算入口页面uv vs. 增量计算入口页面uv）
One-Pass & Mergable。推荐StreamLib。

流式聚合将聚合拆分为LocalReduce
+GlobalReduce以避免热点。

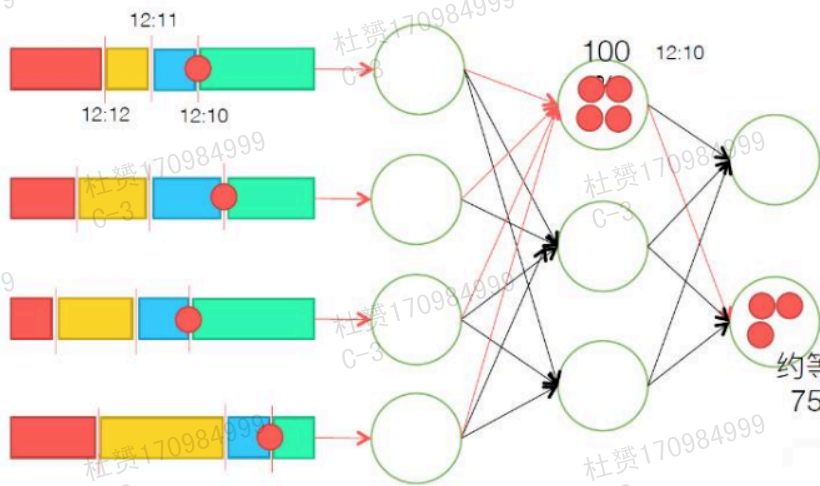
监控系统确定性保证

做监控系统，大家只要做过监控系统底层设施建设的其实都可能碰到这么样一个问题，也就是说那这个问题，其实我在刚开始做鹰眼的时候也是碰到比较痛苦的一个问题，就是说凌晨两点钟突然监控的曲线掉下来了，这时候我应该给运维人员发报警，还是不发报警，为什么我很难做这个抉择，因为我不知道监控的对象真的出了问题，还是监控系统本身的流计算部分出了问题，流计算系统是不是哪里卡住了，还是数据收集通道出现了问题。其实当流计算系统被应用到了监控领域中，它提出了更高的要求，一个叫做“确定性”的要求。“确定性”要求在监控系统里面，是远远大于最终一致性的，所以为了解决这个问题，我们看了一圈现在的流计算引擎提供的解法，实际上没有很好的现成方案去解决这个问题。

我们自己实现了一个齐全度算法来完成确定性的保证。它与 Apache Flink 中采纳的 Snapshot 算法有些近似，它其实是在整个流的过程当中去安插一些屏障 (Barrier)，我们可以做一个假设，在所有流计算数据产生的地方，数据都是有序的。也就是说我们在收集数据的时候，通过第一探测数据源的深度，第二个在数据时间间隔出现交替的时候，比如说从 12:10 跑到 12: 11 的时候去安插一个屏障随着流做传递，流计算过程中保证 FIFO，下游可以把这些屏障聚合起来，这样最终数据落地的时候，我就可以知道数据源当中有多少个屏障已经达到了最终的存储当中。通过这种方式，我们其实可以预测出当一个系统在任意一个时刻它的齐全度是多少，这样的话我们其实可以很容易得到一件事，就是说这个线掉下去了，但是因为在某一个时间点的屏障还没有完全到齐，所以我现在可以预测是因为监控系统卡住了，没有办法给下游的运维人员发报警。因为我知道现在数据还没有收齐，我没有具备确定性的阈值保证，是没有办法去发报警的。这个齐全度的算法，也就是我们在流计算当中，针对监控系统报警场景作出的确定性保证的优化。

相关优化 – 齐全度优化

延迟与准确性SLA



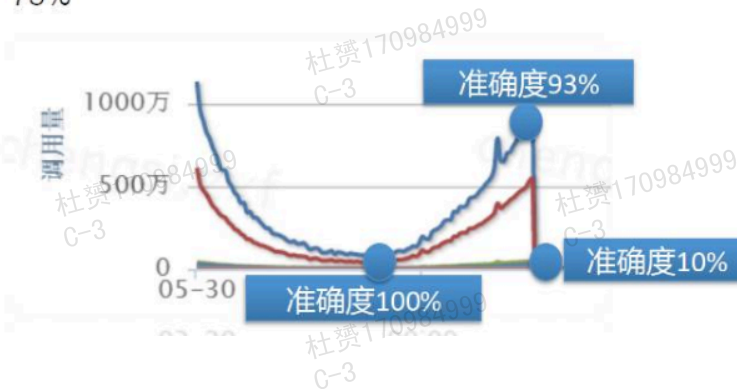
Partition越大，Partition数据越均匀，到达Barrier/全部Barrier越接近于真实齐全度。

产生机制：

基于Snapshot算法。
利用单一数据源局部有序特性。
在业务时间窗口交界处产生。
(Watermark)

用途：

依据任意时间点该Watermark百分比，估算出该时间点的齐全度。



个性化需求

不同的业务数据，会有不同的计算需求。为了满足大家的需求，这里把一些可复用的逻辑变成了“积木块”，让用户按照自己的需求，拼装流计算的 pipeline。其实就是把整个监控系统当中这些模块，进行了抽象和分类，比如说切分、逻辑判断、聚合、持久化、告警，还有一些用户自定义的逻辑，这一块也是借助了这套叫做 Google Blockly 开源可视化的框架，我们对它进行二次开发。用户可以直接在我们的界面上，拼出自己的监控数据处理流程。

用“积木块”搭建数据处理流程 (ARMS，内部名TLog)

分词、切分



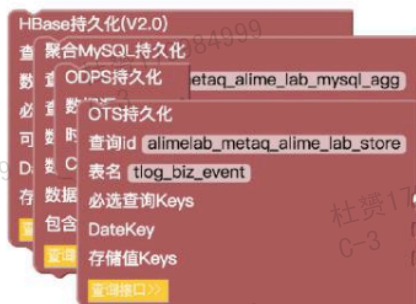
逻辑判断



聚合计算



持久化



告警、消息



Still in Lab



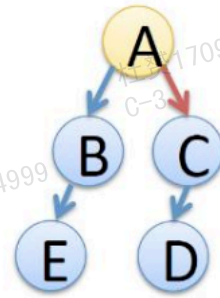
Powered by Google Blockly

定位故障原因

大家在发布的时候，都会盯着日志看，看日志跟以前长得差不多，这次发布好像是没什么问题，换一个人来发布的时候就完全懵了，因为他不知道日志以前长什么样子。所以，我们对日志进行了归纳整理，当发布时如果发现日志和之前归纳的不一样了，再看看最近是否有发布、配置变更、网络故障等事件，以及上下游是否有问题，即可定位故障根本原因。

识别+关联+定位

发布事件 配置变更
网络故障 业务事件



ArchSummit
全球架构师峰会 2017

Geekbang InfoQ

案例

波动前1分钟
分库分表规则配置变更



某核心入口应用
A业务指标出现
小幅波动
应用B卖家中心
日志出现新增异
常堆栈



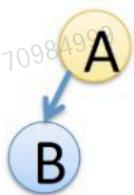
按点关联：B应用接收
到了配置变更推送
按时间线关联：配置变
更发生在指标波动前1
分钟。
按链路关联：A应用强
依赖B应用。



配置变更为引
发指标波动的
高嫌疑原因。



人工验证：
卖家中心新增异常堆
栈为：跨库事务
异常嫌疑确认



系统优化

存储优化



存储层优化

调用链存储

- 阶段一：使用 HBase 存储，Traceld 做 Rowkey
- 阶段二：使用 Hadoop/ODPS 存储
 1. 按 Traceld 的时间戳和哈希值进行分片，分片内按 Traceld 排序
 2. 针对调用中记录的每个列进行针对性压缩，节省存储
- 阶段三：使用分库分表的列式存储 HiStore，按 Traceld 的时间戳和哈希值进行分库分表HiStore 支持列式高压缩比存储，兼容 MySQL 生态，非常适合写多读少的场景。

指标存储

- 基于OpenTSDB的Rowkey Schema
- 移除OpenTSDB的Proxy层
- 流计算中完成降精度
- Co-Processor和预计算（流计算）互补。解决合并时间窗、TTL等多种后计算需求。

从被动报警到主动发现

从被动到主动

被动->主动



我们真的需要存储每一条链路吗？

我们的用户真的理解每一张报表的含义吗？

我们能否比用户更早地发现问题、定位问题？

被动->主动解决思路

识别

- 只存储“可疑的”数据
- 调用返回错误、超时
- 指标离群点、链路离群点、时序异常点

关联

- 只展示与可疑数据有关联的报表
- 点(事件)与面（指标）
- 上游与下游

定位

- 根据关联关系，将“果”关联到“因”

Ⓜ 仅供内部使用，未经授权，切勿外传